

A Mathematical Tour of Lane Detection and Autonomous Steering Control

A pedagogical companion to the project for a CS student

David Tovmasyan
david.tovmasyan@connectto.com

May 2026

Abstract

This document is the long-form, mathematically explicit companion to the project “Real-Time Lane Detection and Autonomous Steering”. Where the short report focused on the experimental story, this document tells the *pedagogical* story: every formula behind every stage of every method is derived, illustrated with worked numerical examples, and tied back to the actual lines of Python in the repository. The reader is assumed to be a third-year CS student comfortable with linear algebra and basic calculus, but not necessarily familiar with computer vision or feedback control. By the end of the document, you should be able to (a) reimplement each detector from scratch, (b) explain why the proposed bird’s-eye-view polynomial-fit detector outperforms a Hough-line baseline on curved tracks, and (c) analyse a PID and a Stanley controller as feedback laws on the bicycle model.

Contents

1	Introduction and Motivation	2
2	Mathematical Preliminaries	4
2.1	An image is a function	4
2.2	RGB to grayscale	4
2.3	The HSV colour space	5
2.4	2D convolution	5
2.5	The Gaussian kernel	6
2.6	Image gradients: the Sobel operator	7
2.7	Canny edge detection	7
3	The Driving Simulator	7
3.1	Track geometry: the Catmull–Rom spline	8
3.2	Curvature of a parametric curve	8
3.3	Bicycle kinematic model	9
3.4	The pinhole camera	9
3.4.1	Focal length from field of view	9
3.4.2	Camera frame and pitch rotation	10
3.4.3	The full FPV render	10
4	The Lane-Detection Sub-problem	11
4.1	Inputs and outputs	11
4.2	Ground truth available in the simulator	11
4.3	Preview of the three detectors	11
5	Detector 1 – The Centroid Baseline	12
5.1	Image moments and centroids	12
5.2	The road-colour mask	12
5.3	Worked example: centroid of a tiny mask	12
5.4	Why the centroid fails structurally	13

6	Detector 2 – The Hough-Line Pipeline	13
6.1	Pipeline overview	13
6.2	The Hough transform: polar parameterisation	14
6.2.1	The voting procedure	14
6.3	Probabilistic Hough	14
6.4	Slope-based classification	15
6.5	Length-weighted averaging	15
6.6	Centre offset and temporal smoothing	16
6.7	What it looks like	16
6.8	Why the Hough detector misses curves	16
7	Detector 3 – Polynomial Fit in Bird’s-Eye View	17
7.1	Motivation: why straight lines are the wrong model	17
7.2	The lane-pixel mask	17
7.3	Inverse perspective mapping	18
7.4	Histogram base detection	19
7.5	Sliding-window search	19
7.6	Polynomial regression	20
7.7	The look-ahead offset (the key idea)	21
7.8	Lane reprojection	22
7.9	Putting it all together	22
8	Steering Controllers	23
8.1	The feedback control loop	23
8.2	PID – the baseline	23
8.3	Stanley – the perception-free oracle	24
8.4	Adaptive speed control	25
9	Metrics for Quantitative Evaluation	25
9.1	Lateral error	26
9.2	Heading error	26
9.3	Lane IoU	26
9.4	Detection confidence	26
9.5	Inference time and FPS	27
9.6	Off-track rate	27
9.7	Lap completion	27
10	Experimental Results and Analysis	27
10.1	Setup	27
10.2	Per-track method comparison	27
10.3	Per-frame trace on a winding track	29
10.4	Trajectories overlay	29
10.5	Robustness to camera noise	30
10.6	Compute cost	31
10.7	Failure cases	31
11	Conclusion and Future Work	32

1 Introduction and Motivation

Lane keeping is, by a comfortable margin, the most widely deployed function in modern advanced driver-assistance systems (ADAS). Every mass-market car with “lane-keep assist” or “lane-centering” on its spec sheet contains – somewhere behind a windshield-mounted camera – some version of the pipeline this document explains. The economics are unromantic but compelling: a \$20 image sensor, a \$2 microcontroller, and a few thousand lines of code can subtract the most common cause of single-vehicle highway crashes (drift out of lane due to inattention or drowsiness). For the same reason, lane detection has become a near-universal entry point into autonomy: it is geometrically simple, visually intuitive, and tractable on a CPU. It is therefore an ideal classroom problem – a tiny slice of self-driving that nevertheless exercises camera calibration, image processing, edge detection, line fitting, polynomial regression, vehicle dynamics, and feedback control.

The perception–planning–control split

Autonomous-driving stacks are almost always organised into three layers, executed once per frame in a fixed pipeline:

1. **Perception** – read the camera (and, in real cars, radar, LiDAR, IMU), and produce a structured description of the world: “the left lane boundary is the curve $x = ay^2 + by + c$ ”.
2. **Planning** – from that description, decide *where* the car should be in the next few hundred milliseconds: “stay centred between the boundaries with a small look-ahead”.
3. **Control** – compute the steering and throttle commands that drive the vehicle toward that target, given the vehicle’s current dynamic state.

In this project the same split is visible: `lane_methods.py` implements perception, the lane-centre target is computed implicitly as “the midpoint of the two boundary curves”, and `controller.py` implements two control laws (PID and Stanley). The simulator – the `camera.py` renderer, the `track.py` spline and the `vehicle.py` bicycle model – replaces the physical world.

Why classical CV rather than deep learning?

A naive reading of the recent literature might suggest the problem is “solved” by learning. State-of-the-art lane detectors – LaneNet, SCNN, PolyLaneNet, Ultra-Fast-Lane-Detection – learn an end-to-end map from pixels to lane parameters and dominate every public benchmark (TuSimple, CULane, LLAMAS). Why then would anyone deploy a hand-written Canny + Hough pipeline in 2026?

- **Interpretability.** Every step of the classical pipeline has a closed-form mathematical definition. If the car misbehaves, the failure can be traced to a specific stage: the ROI was wrong, the threshold was too high, the Hough binning was too coarse. With a black-box CNN this kind of forensic analysis is far harder.
- **Compute footprint.** The full pipeline in this project runs at > 1000 FPS on a single CPU thread; a learned segmenter typically needs a GPU or NPU. For low-end ADAS modules without an accelerator, classical CV is the only option.
- **No training data, no labelling, no domain shift.** A classical detector contains no learned weights, so there is no train/test distribution mismatch. Its assumptions are visible (“edges of constant gradient form lines”), so we can predict *a priori* where it will fail.
- **Educational clarity.** For a course project, every line of `HoughDetector.detect` is a piece of mathematics we can derive. Every line of a UNet is opaque.

The trade-off is equally clear: classical detectors assume a clean, high-contrast world. They struggle with shadows, weathered paint, crowded roads, and – as we will see in detail – with curves. The question this project answers is therefore very specific: *given that the textbook Hough lane detector clearly fails on curves, how much of that gap can be closed by a slightly less classical method that still avoids deep learning?* The proposed answer – a bird’s-eye sliding-window quadratic fit – reduces RMS lateral error by 34 % on average across five tracks, while remaining interpretable and real-time.

Roadmap of this document

This document is the first half of a long-form, pedagogical companion to the project. The short report (`presentation/report.tex`) gives the experimental punch line in three pages; this document gives every derivation, every worked example, and every line of code behind the punch line.

- Section 2 covers the *mathematical preliminaries*: images as functions, colour spaces, convolution, Gaussian smoothing, the Sobel operator, and the four steps of the Canny edge detector. If you have not seen these before, this is where to spend the most time.
- Section 3 dissects the *simulator*: the Catmull–Rom spline that defines each track, the discrete curvature formula, the bicycle kinematic model, and the pinhole camera that produces the first-person image the detectors see.

- Section 4 formalises the lane-detection *sub-problem*: what the detectors are asked to compute, and what ground truth is available for evaluation.
- Section 5 works through the first detector – the *centroid baseline* – and explains, with image moments, why it cannot possibly succeed.
- Section 6 works through the second detector – the *Hough-line pipeline* – in full mathematical detail, including the polar parameterisation of a line, the probabilistic Hough variant used by OpenCV, slope classification, length-weighted averaging, and temporal exponential smoothing.

The second half of the document (a separate file) picks up with the inverse-perspective mapping and the proposed polynomial-fit detector, then continues into vehicle control and the experimental evaluation.

2 Mathematical Preliminaries

Before we can speak meaningfully about “detecting lanes” we need to agree on what an image *is*, and on the handful of operations we will perform on it. This section is deliberately slow: a third-year CS student who has never taken a CV class should be able to follow every formula and reproduce every numerical example by hand.

2.1 An image is a function

Definition 2.1 (Discrete image). A *grayscale image* of width W and height H is a function

$$I : \{0, \dots, W-1\} \times \{0, \dots, H-1\} \rightarrow \{0, \dots, 255\},$$

mapping integer pixel coordinates (u, v) to an 8-bit intensity. A *colour image* has the same domain but a three-component codomain, $I : (u, v) \mapsto (R, G, B) \in \{0, \dots, 255\}^3$.

Two conventions must be stated explicitly because they trip up almost every newcomer:

1. The origin is at the **top-left** of the image. The first axis u increases to the right; the second axis v increases *downward*. This is opposite to the convention you learned in calculus class, where y usually points up.
2. In our simulator the world coordinate (X, Y) uses the same screen-down convention as the image (because everything is drawn into a Pygame window whose origin is also top-left). Be careful not to confuse image pixels and world pixels: the camera’s 640×480 frame is one coordinate system; the 1400×800 Pygame window is another.

The camera in this project produces images of size $W = 640$, $H = 480$ (set in `config.py`: `CAMERA_IMG_W=640`, `CAMERA_IMG_H=480`). A single frame is a NumPy array of shape $(480, 640, 3)$ with `dtype=uint8`.

2.2 RGB to grayscale

The first thing every classical detector in this project does is throw away colour: it converts the RGB image to a single-channel grayscale image. The standard conversion follows ITU-R Recommendation BT.601 (“Rec. 601”), originally designed so that the luminance channel of analogue colour television matches what a human eye perceives as brightness:

$$Y = 0.299 R + 0.587 G + 0.114 B. \tag{1}$$

The coefficients are not arbitrary: the human eye is much more sensitive to green than to red, and least sensitive to blue. A simple unweighted average $(R + G + B)/3$ would make red and green grass look almost identical even though they are visually obviously different.

Example 2.1 (Grayscale of a road pixel). Our simulator paints the road in $\text{RGB} = (64, 68, 74)$ (see `config.py:ROAD_SURFACE`). Applying (1):

$$Y = 0.299 \cdot 64 + 0.587 \cdot 68 + 0.114 \cdot 74 \approx 19.14 + 39.92 + 8.44 \approx 67.5.$$

So the road has roughly intensity 68 on a $[0, 255]$ scale – a fairly dark grey, but well above black. Compare to grass $\text{RGB} = (34, 52, 30)$: $Y \approx 10.2 + 30.5 + 3.4 = 44.1$. The road is about 50% brighter than the grass; that contrast is what lets the Canny edge detector discover the road boundary.

In OpenCV the call is `cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)` and it is the very first line of every `detect()` method in this project (e.g. `lane_methods.py:HoughDetector.detect`, line 203).

2.3 The HSV colour space

Sometimes we want to threshold by colour rather than brightness – for example, to find the yellow centre dashes (`LANE_DASH_COLOR = (200, 200, 80)`) regardless of how brightly lit they are. RGB is awkward for this because “yellow” is a curve in 3-space rather than an axis; HSV separates the perceptual axes:

- **Hue** $H \in [0, 360^\circ)$: which colour. Red ≈ 0 , yellow ≈ 60 , green ≈ 120 , cyan ≈ 180 , blue ≈ 240 , magenta ≈ 300 . (OpenCV stores H scaled to $[0, 180)$ so it fits in a `uint8`.)
- **Saturation** $S \in [0, 1]$: how colourful. $S = 0$ is pure grey; $S = 1$ is fully saturated.
- **Value** $V \in [0, 1]$: how bright. $V = 0$ is black; $V = 1$ is the brightest version of the hue.

The conversion is piecewise. Given $R, G, B \in [0, 1]$, let $M = \max(R, G, B)$, $m = \min(R, G, B)$ and $C = M - m$. Then $V = M$, $S = C/M$ (if $M > 0$, else 0), and

$$H = 60^\circ \cdot \begin{cases} \frac{G - B}{C} \bmod 6, & M = R, \\ \frac{B - R}{C} + 2, & M = G, \\ \frac{R - G}{C} + 4, & M = B. \end{cases}$$

Example 2.2 (HSV of the road pixel). Take the road colour $(R, G, B) = (64, 68, 74)$. Normalise to $[0, 1]$: $(0.251, 0.267, 0.290)$. Then $M = 0.290$ (the blue channel), $m = 0.251$, $C = 0.039$.

$$\begin{aligned} V = 0.290 &\Rightarrow V \cdot 255 \approx 74, \\ S = C/M = 0.039/0.290 &\approx 0.135 \Rightarrow S \cdot 255 \approx 34, \\ H = 60^\circ \left(\frac{R-G}{C} + 4 \right) &= 60^\circ \left(\frac{-0.016}{0.039} + 4 \right) \approx 60^\circ \cdot 3.59 \approx 215^\circ. \end{aligned}$$

So the road sits at hue $\approx 215^\circ$ (a slightly cool blue-grey, which is correct – the colour is barely off neutral), low saturation $\approx 34/255$, and moderate value $\approx 74/255$. The `CentroidDetector` uses precisely this signature to threshold the road: `(s < 40) & (v > 40) & (v < 110)` (line 122 of `lane_methods.py`). The chosen bounds bracket exactly the (S, V) values we just computed.

2.4 2D convolution

Everything we do to the image after grayscale conversion – blurring, edge detection, gradient estimation – is a *linear filter*, and every linear filter can be written as a *convolution* between the image I and a small *kernel* K .

Definition 2.2 (2D discrete convolution). Given image I and kernel K of size $(2k+1) \times (2k+1)$, the convolution $(I * K) : \mathbb{Z}^2 \rightarrow \mathbb{R}$ is

$$(I * K)(u, v) = \sum_{i=-k}^k \sum_{j=-k}^k I(u-i, v-j) K(i, j). \quad (2)$$

The minus signs in (2) are not a typo: a true convolution flips the kernel before sliding it. In practice many image-processing libraries (OpenCV included) actually compute the *correlation* $(I \star K)(u, v) = \sum_{i,j} I(u+i, v+j)K(i, j)$ – with no flip – and call it convolution. For *symmetric* kernels (Gaussian, mean, Sobel-magnitude¹) the two operations agree, so the pedantry rarely bites.

Example 2.3 (A 3×3 box blur). Consider a tiny grayscale image and a 3×3 averaging kernel:

$$I = \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{pmatrix}, \quad K = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

At the centre pixel $(1, 1)$ the convolution is the mean of all nine values, $(10 + 20 + \dots + 90)/9 = 450/9 = 50$. So $(I \star K)(1, 1) = 50$. At a boundary pixel we have to decide how to handle the missing neighbours; OpenCV by default replicates the edge value, but the convolution formula is identical otherwise.

2.5 The Gaussian kernel

Random pixel noise (and noise added in our experiments via `evaluate.py:_add_noise`) is the enemy of edge detection: a single bright spike inside an otherwise smooth patch produces a strong gradient that no edge actually exists. The standard remedy is to blur the image with a *Gaussian kernel* before computing gradients:

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \quad (3)$$

The function is rotationally symmetric and integrates to 1 over $\mathbb{R}^2$; in discrete form we sample (3) at integer offsets, then renormalise so the sampled values sum exactly to 1 (to preserve average brightness).

Example 2.4 (5×5 Gaussian with $\sigma = 1$). Sampling (3) at $(x, y) \in \{-2, -1, 0, 1, 2\}^2$ with $\sigma = 1$ gives the unnormalised values

$$\tilde{G} = \begin{pmatrix} 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0219 & 0.0983 & 0.1620 & 0.0983 & 0.0219 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \end{pmatrix}.$$

The entries sum to about 0.9989, so renormalising hardly changes them. A common integer approximation (used in many introductory tutorials) is

$$G = \frac{1}{273} \begin{pmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{pmatrix},$$

whose centre value $41/273 \approx 0.150$ is close to the exact 0.162. Our project calls `cv2.GaussianBlur(gray, (5, 5), 0)` (line 204 of `lane_methods.py`); passing $\sigma = 0$ tells OpenCV to derive σ from the kernel size as $\sigma = 0.3((k-1)/2 - 1) + 0.8 \approx 1.1$, which is essentially the kernel we just built.

A larger σ blurs more aggressively, removes finer noise, but also smears out genuine edges. The 5×5 kernel with $\sigma \approx 1$ is a good default for 640×480 images: it attenuates pixel-scale noise without dissolving lane markings.

¹Sobel S_x, S_y are antisymmetric, but the sign convention is folded into the definition; what matters is being consistent.

2.6 Image gradients: the Sobel operator

An edge is, intuitively, a place where the image brightness changes rapidly. Mathematically: a place where the spatial derivative ∇I has a large magnitude. To approximate ∇I on a discrete grid we convolve with the Sobel kernels:

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}. \quad (4)$$

Each kernel is the discrete tensor product of a smoothing kernel $(1, 2, 1)$ in one direction with a central-difference derivative $(-1, 0, 1)$ in the other; you can think of S_x as “smooth vertically then differentiate horizontally”. The output channels $G_x = I * S_x$ and $G_y = I * S_y$ are the estimated partial derivatives; gradient magnitude and direction follow:

$$|G| = \sqrt{G_x^2 + G_y^2}, \quad \alpha = \text{atan2}(G_y, G_x). \quad (5)$$

The `PolyFitDetector` uses S_x explicitly to find vertical-ish lane markings (line 406 of `lane_methods.py`: `cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)`). The Hough detector does not call Sobel directly because the Canny edge detector (below) does it internally.

2.7 Canny edge detection

`cv2.Canny` reduces a grayscale image to a binary edge map: pixel (u, v) is 255 if Canny believes there is an edge there, and 0 otherwise. The Canny algorithm (Canny 1986) is composed of four steps that are all worth understanding individually:

1. **Gaussian blur.** As described above, smooth the image to suppress single-pixel noise. Without this step the derivatives in step 2 would amplify every speckle.
2. **Sobel gradients.** Compute G_x, G_y and the gradient magnitude $|G|$ from (5). The magnitude image already looks like an edge map – but it is too thick (real edges are typically several pixels wide because the gradient is non-zero across the whole transition), and includes weak responses everywhere.
3. **Non-maximum suppression.** For each pixel, check whether its gradient magnitude is larger than that of its two immediate neighbours along the gradient direction α . If not, zero it out. This thins the edge map to single-pixel-wide ridges, which is what we need for line fitting.
4. **Double-threshold hysteresis.** A high threshold T_H marks “strong” edges; a low threshold T_L marks “candidate” edges. A candidate edge is retained iff it is connected (8-neighbourhood) to a strong edge. This *hysteresis* is what makes Canny robust: an edge that peaks weakly but is part of a long contour survives, while isolated weak edges (noise) are discarded.

This project uses `cv2.Canny(blurred, 50, 150)` with $T_L = 50, T_H = 150$ (`config.py:CANNY_LOW=50, CANNY_HIGH=150`). The classical heuristic is $T_H : T_L$ between 2:1 and 3:1; we use exactly 3:1. These specific numbers were tuned against the simulator’s road colours: 50 is just above the noise floor of the rendered frame, and 150 comfortably excludes everything except the sharp lane-boundary edges.

Intuition

An edge is where the image brightness changes fast in some direction. Canny finds those places, thins the response so each edge is exactly one pixel wide, and then keeps only the ones long enough (via hysteresis) to be a real contour rather than a stray noise spike. Everything else in the lane-detection pipeline operates on this thinned binary picture.

3 The Driving Simulator

The simulator is small but it is doing a surprising amount of work: generating a smooth closed-loop track, computing its curvature, simulating a kinematic bicycle vehicle, and rendering the world through a virtual pinhole camera. We treat each piece in turn.

3.1 Track geometry: the Catmull–Rom spline

A track in `config.py` is a list of n waypoints $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{n-1} \in \mathbb{R}^2$ (e.g. 12 for the oval, 21 for Red Bull Ring). Connecting them with straight line segments would give a polygon, not a track – the resulting heading changes would be discontinuous, the steering would saturate at every waypoint, and the simulation would look terrible. We need a *smooth* curve that passes through every waypoint.

Definition 3.1 (Catmull–Rom spline). Given four control points $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$, the Catmull–Rom spline segment between $\mathbf{p}_1$ and $\mathbf{p}_2$ is

$$\begin{aligned} \mathbf{q}(t) = & \frac{1}{2} \left[(2\mathbf{p}_1) + (-\mathbf{p}_0 + \mathbf{p}_2) t \right. \\ & + (2\mathbf{p}_0 - 5\mathbf{p}_1 + 4\mathbf{p}_2 - \mathbf{p}_3) t^2 \\ & \left. + (-\mathbf{p}_0 + 3\mathbf{p}_1 - 3\mathbf{p}_2 + \mathbf{p}_3) t^3 \right], \end{aligned} \quad (6)$$

for $t \in [0, 1]$. The spline *passes through* $\mathbf{p}_1$ at $t = 0$ and $\mathbf{p}_2$ at $t = 1$, while $\mathbf{p}_0$ and $\mathbf{p}_3$ control the tangents at the endpoints.

To check the interpolation property, substitute $t = 0$ into (6): the t^1, t^2, t^3 terms vanish, leaving $\mathbf{q}(0) = \frac{1}{2} \cdot 2\mathbf{p}_1 = \mathbf{p}_1$. Likewise $t = 1$ gives $\mathbf{q}(1) = \mathbf{p}_2$. Differentiating,

$$\mathbf{q}'(0) = \frac{1}{2}(\mathbf{p}_2 - \mathbf{p}_0), \quad \mathbf{q}'(1) = \frac{1}{2}(\mathbf{p}_3 - \mathbf{p}_1),$$

so the tangent at $\mathbf{p}_1$ is parallel to the chord $\mathbf{p}_2 - \mathbf{p}_0$ – the segment “aims through” the next waypoint. The same tangent value is shared by the adjacent segment, so the assembled curve is C^1 -continuous (no kinks in heading) through every waypoint. That is exactly what a drivable track needs.

In code (`track.py:Track._interpolate`, lines 31–52), the spline is sampled at `TRACK_SAMPLES_PER_SEGMENT = 60` values of $t \in [0, 1]$ per segment, looping the index modulo n so the track closes back to its first waypoint. For the 12-waypoint oval this gives $12 \times 60 = 720$ centreline samples, dense enough that finite-difference tangents and curvatures are well-behaved.

3.2 Curvature of a parametric curve

Once we have the centreline as a parametric curve $\mathbf{c}(s) = (x(s), y(s))$, we often want its *curvature* – how sharply it is bending. Two reasons: (i) the adaptive-speed controller slows the car before curves, so it needs to know which sections are sharp; (ii) the Hough detector’s main failure mode is that it cannot represent curvature, so understanding how curved each track is tells us where to expect Hough to break.

Definition 3.2 (Curvature of a 2D parametric curve). For a curve $(x(s), y(s))$ with finite first and second derivatives, the (signed) curvature is

$$\kappa(s) = \frac{x'(s)y''(s) - y'(s)x''(s)}{(x'(s)^2 + y'(s)^2)^{3/2}}. \quad (7)$$

The simulator uses the unsigned version $|\kappa|$ (it only cares about “how curvy”, not “which way”), implemented in `track.py:Track._compute_curvature` on lines 68–103 via central differences:

$$\dot{x}_i \approx \frac{x_{i+1} - x_{i-1}}{2}, \quad \ddot{x}_i \approx x_{i+1} - 2x_i + x_{i-1},$$

and similarly for y . The grid spacing is absorbed into the $(\cdot)^{3/2}$ factor in the denominator, which is why the formula in code has no explicit Δs .

Units. If (x, y) are in pixels and the parameter s is also in pixels, then x' is dimensionless, x'' has units of $1/\text{px}$, and the numerator of (7) has units of $1/\text{px}$. The denominator $(x'^2 + y'^2)^{3/2}$ is dimensionless. So κ has units of $1/\text{px}$, which equals $1/R$ for a circle of radius R – exactly what curvature should be.

After computing κ point-by-point, the simulator smooths the result with a 15-point rolling average (lines 100–102) to suppress spline-sampling jitter; only then is it used by the `SpeedController`.

3.3 Bicycle kinematic model

The vehicle's state at time t is the triple $(x(t), y(t), \theta(t))$ – position and heading – plus a scalar forward speed v and a steering angle δ . We need a differential equation that says how (x, y, θ) evolves as a function of v and δ .

The *bicycle kinematic model* makes one simplifying assumption: the left and right wheels of each axle are collapsed into a single wheel on the centreline, so the car becomes a bicycle. Two further assumptions follow from this:

1. **No side slip.** Each wheel's velocity vector is aligned with the direction the wheel is pointing. The rear wheel points along the body's heading θ ; the front wheel points along $\theta + \delta$.
2. **Rear-axle reference.** We track the position of the rear axle. This is convenient because the rear wheel's velocity equals the body's velocity in the body frame.

Under these assumptions the velocity of the rear axle is just $(v \cos \theta, v \sin \theta)$. To find how fast θ changes, observe that during a small time dt the rear and front axles each move forward by $v dt$, but the front axle moves along heading $\theta + \delta$ while the rear moves along θ . The front axle must therefore swing around the rear by an angle $d\theta$ such that the front-axle displacement perpendicular to the body, $v dt \sin \delta$, equals $L d\theta$ (where L is the wheelbase). Hence $\dot{\theta} = (v \sin \delta) / L \cdot (1 / \cos \delta) = (v / L) \tan \delta$ once you keep the projection along the body axis consistent.

The full system is therefore

$$\boxed{\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \frac{v}{L} \tan \delta.} \quad (8)$$

In code (`vehicle.py:Vehicle.update`, lines 19–31), this is integrated by forward Euler: $x \leftarrow x + v \cos \theta \Delta t$, $y \leftarrow y + v \sin \theta \Delta t$, $\theta \leftarrow \theta + (v / L) \tan \delta \Delta t$. The steering angle is first clamped to $\pm 40^\circ$ (`config.py:MAX_STEERING_ANGLE`).

Example 3.1 (Instantaneous turn radius). With $L = 25$ px, $\delta = 20^\circ$ and $v = 100$ px/s, the bicycle's instantaneous turn radius is

$$R = \frac{L}{\tan \delta} = \frac{25}{\tan 20^\circ} \approx \frac{25}{0.364} \approx 68.7 \text{ px},$$

and the angular velocity is

$$\dot{\theta} = \frac{v}{R} = \frac{100}{68.7} \approx 1.46 \text{ rad/s},$$

or about $83^\circ/\text{s}$ – the car would rotate 360° in roughly 4.3 s. Equivalently, $\dot{\theta} = (v / L) \tan \delta = (100 / 25) \cdot 0.364 = 1.46 \text{ rad/s}$, matching (8).

3.4 The pinhole camera

The camera in `camera.py` takes world points (X, Y) – on the ground plane in front of the vehicle – and produces image pixel coordinates (u, v) . It is the most important piece of geometry in the whole project, because every detector is fed the output of this projection.

3.4.1 Focal length from field of view

A pinhole camera with image width W and horizontal field of view FoV satisfies

$$\tan(\text{FoV}/2) = \frac{W/2}{f} \iff f = \frac{W/2}{\tan(\text{FoV}/2)}. \quad (9)$$

With $W = 640$ and $\text{FoV} = 70^\circ$ (`config`),

$$f = \frac{320}{\tan(35^\circ)} \approx \frac{320}{0.7002} \approx 457.0 \text{ px}.$$

This is the value computed on line 14 of `camera.py`: `self.focal = (self.w / 2) / math.tan(fov_rad / 2)`.

3.4.2 Camera frame and pitch rotation

The camera is mounted at height $h = 50$ px above the ground, pitched down by $\phi = 18^\circ$. Define a *vehicle-local* frame whose origin is on the ground directly beneath the camera, $+Z$ pointing forward along the vehicle's heading, $+X$ pointing right, $+Y$ pointing down. A ground point in front of the car has coordinates $(X_{\text{lat}}, h, Z_{\text{fwd}})$ – “ h ” because the ground is h pixels *below* the camera, which is the $+Y$ direction.

The camera frame after pitch is obtained by rotating around the X axis by ϕ :

$$\begin{pmatrix} X_c \\ Y_c \\ Z_c \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{pmatrix} \begin{pmatrix} X_{\text{lat}} \\ h \\ Z_{\text{fwd}} \end{pmatrix}. \quad (10)$$

Carrying out the multiplication explicitly,

$$\begin{aligned} X_c &= X_{\text{lat}}, \\ Y_c &= h \cos \phi - Z_{\text{fwd}} \sin \phi, \\ Z_c &= h \sin \phi + Z_{\text{fwd}} \cos \phi. \end{aligned} \quad (11)$$

These three lines are exactly lines 58–60 of `camera.py`. Finally the perspective division gives image coordinates:

$$u = f \frac{X_c}{Z_c} + c_x, \quad v = f \frac{Y_c}{Z_c} + c_y, \quad (12)$$

where $(c_x, c_y) = (W/2, H/2) = (320, 240)$ is the principal point. A point is visible iff $Z_c > 2$ (just in front of the camera; see line 63 of `camera.py`).

Example 3.2 (Projecting a ground point). Take a point 200 px directly ahead of the camera on the ground: $X_{\text{lat}} = 0$, $h = 50$, $Z_{\text{fwd}} = 200$. With $\phi = 18^\circ$, $\cos \phi \approx 0.951$, $\sin \phi \approx 0.309$:

$$\begin{aligned} X_c &= 0, \\ Y_c &= 50 \cdot 0.951 - 200 \cdot 0.309 \approx 47.55 - 61.80 \approx -14.25, \\ Z_c &= 50 \cdot 0.309 + 200 \cdot 0.951 \approx 15.45 + 190.20 \approx 205.65. \end{aligned}$$

With $f \approx 457$,

$$u = 457 \cdot 0/205.65 + 320 = 320, \quad v = 457 \cdot (-14.25)/205.65 + 240 \approx -31.66 + 240 = 208.3.$$

So this point projects to roughly $(320, 208)$ – horizontally centred (it was straight ahead) and just above the image vertical centre 240, which makes sense: with the camera pitched down, a point on the horizon would lie below 240, while a point only 200 px ahead is nearer than the horizon and projects higher.

3.4.3 The full FPV render

`FPVCamera.render` (lines 79–141 of `camera.py`) puts the above together for an entire frame:

1. Copy a pre-computed vertical sky gradient (lines 24–32).
2. Compute the horizon row analytically: as $Z_{\text{fwd}} \rightarrow \infty$, equation (11) gives $Y_c \rightarrow -Z_{\text{fwd}} \sin \phi$ and $Z_c \rightarrow Z_{\text{fwd}} \cos \phi$, so $v_{\text{horizon}} = f \cdot (-\sin \phi / \cos \phi) + c_y = -f \tan \phi + c_y$. Fill everything below that with grass.
3. Ask the track for the left, right and centre boundary points within `CAMERA_LOOK_AHEAD = 350` px of the vehicle (`Track.get_road_ahead`).
4. Project those points through (11) and (12), drop the ones behind the camera, fill the polygon between left and right boundaries with the road colour `ROAD_SURFACE = (64, 68, 74)`.
5. Draw the solid white boundary lines and the dashed yellow centre line on top.

The result is a 640×480 RGB image – the same shape and content as if a real camera were filming a real road – which is then handed off to the lane detector. There is no real-world data anywhere in the pipeline; the simulator is the world.

Intuition

You can think of the camera as the *inverse* of the inverse-perspective (IPM) warp we will meet in Section 7. The camera takes world points and projects them onto the image plane; the IPM takes image pixels and projects them back onto the ground plane. The polyfit detector explicitly inverts what the camera does.

4 The Lane-Detection Sub-problem

We have now described the world the detectors live in. What exactly are they asked to do?

4.1 Inputs and outputs

Every lane detector in this project implements the same interface (`lane_methods.py: LaneDetectorBase`, line 81). It takes a single input – the current camera frame as an RGB NumPy array of shape $(H, W, 3) = (480, 640, 3)$ – and returns a `Detection` object with the following fields:

- **center_offset**: signed horizontal offset (in camera pixels) of the detected lane centre from the image centre. Positive means the lane centre is to the right of the camera, so the car must steer right to centre itself.
- **lookahead_offset**: the same quantity but evaluated further up the image (i.e. further ahead in world terms). Equal to **center_offset** for line-based detectors; meaningfully different for the polyfit detector on curves.
- **confidence**: a scalar in $[0, 1]$. The Hough detector returns 1 if both boundaries were found, 0.5 if only one, and 0 if none.
- **pred_mask**: a binary mask of the predicted lane polygon, used by the evaluator to compute intersection-over-union against the ground-truth road mask.
- **annotated**: an annotated copy of the input image (for the UI; not used by the controller).

4.2 Ground truth available in the simulator

Because the world is synthetic, we have access to two kinds of ground truth that no real camera could ever provide:

1. **Signed lateral error.** `Track.get_nearest_index` (lines 154–164 of `track.py`) returns, for the vehicle’s current (x, y) , the nearest centreline index and the signed perpendicular distance to the centreline. This is the *true* lateral error $e_{\perp}$ – what every detector is trying to estimate.
2. **Ground-truth lane polygon.** The function `evaluate.py: _gt_lane_mask` (lines 68–84) re-projects the simulator’s true road boundary polygon through the same pinhole camera used to render the input image, producing a $H \times W$ binary mask of where the road *really* is in the camera frame. The Lane-IoU metric is then $\text{IoU} = |\text{pred} \cap \text{gt}| / |\text{pred} \cup \text{gt}|$ – a single scalar in $[0, 1]$ that summarises both the position and the shape of the predicted lane.

Because all three detectors return a **pred_mask** of the same shape, they can be compared on the same IoU scale even though they use very different internal representations.

4.3 Preview of the three detectors

Three detectors are implemented, in increasing order of sophistication:

- **Centroid (Section 5).** HSV-threshold the road, take the centroid of the resulting blob. Deliberately simple; serves as a lower bound.

- **Hough (Section 6).** The textbook Canny + Hough pipeline. Fits a single straight line per lane boundary. Works on straights, struggles on curves.
- **Polyfit (Section 7, in Part 2).** Inverse-perspective warp followed by a sliding-window quadratic fit. Can model curves explicitly.

5 Detector 1 – The Centroid Baseline

The `CentroidDetector` (lines 95–171 of `lane_methods.py`) is the simplest possible thing one could try. It is included specifically so that we have a transparent lower bound for the comparison.

5.1 Image moments and centroids

Definition 5.1 (Image moments). The *spatial moments* of an image $I(u, v)$ are

$$m_{pq} = \sum_u \sum_v u^p v^q I(u, v). \quad (13)$$

For a binary mask, $I(u, v) \in \{0, 1\}$, so m_{00} is just the number of foreground pixels, m_{10} is the sum of their u -coordinates, and m_{01} is the sum of their v -coordinates. The *centroid* of the foreground region is

$$\bar{u} = \frac{m_{10}}{m_{00}}, \quad \bar{v} = \frac{m_{01}}{m_{00}}. \quad (14)$$

OpenCV computes these via `cv2.moments(masked, binaryImage=True)` (line 137 of `lane_methods.py`).

5.2 The road-colour mask

The simulator paints the road in RGB= (64, 68, 74), a flat dark grey. In HSV that pixel sits at $(H, S, V) \approx (215^\circ, 34, 74)$ as we computed in the worked example above. The detector therefore takes “the road” to be the set of pixels with low saturation ($S < 40$) and moderate value ($40 < V < 110$):

```
hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
s = hsv[:, :, 1]
v = hsv[:, :, 2]
road = ((s < 40) & (v > 40) & (v < 110)).astype(np.uint8) * 255
```

The bounds are deliberately loose enough to catch the road’s exact HSV signature while excluding the green grass ($H \approx 110^\circ$, $S \approx 100$, $V \approx 52$ – the high saturation kills it) and the sky ($V \approx 28$ to $V \approx 110$ depending on row, but mostly darker and bluer; some marginal sky pixels do leak through, which is one of the failure modes we will analyse below). A trapezoidal ROI (the same one used by the Hough detector for fairness) restricts the mask to the bottom-centre region of the image where the road plausibly appears.

5.3 Worked example: centroid of a tiny mask

Example 5.1 (Centroid of a 5×5 binary mask). Suppose the masked ROI contains the following 5×5 block (with 0 = background, 1 = foreground):

$$M = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix},$$

indexing (u, v) with u the column $\in \{0, \dots, 4\}$ and v the row $\in \{0, \dots, 4\}$.

The number of foreground pixels is $m_{00} = 9$. Summing column indices over the foreground:

$$m_{10} = \sum_{(u,v) \in M} u = 2 + (1 + 2 + 3) + (1 + 2 + 3) + 2 + 2 = 18,$$

and summing row indices:

$$m_{01} = \sum_{(u,v) \in M} v = 0 + (1 + 1 + 1) + (2 + 2 + 2) + 3 + 4 = 16.$$

So the centroid is at $(\bar{u}, \bar{v}) = (18/9, 16/9) = (2.0, 1.78)$ – exactly at the central column $u = 2$, slightly above the middle row, which matches the visible mass distribution (the cross is heavier on top).

5.4 Why the centroid fails structurally

The centroid would work *if* the visible road blob were symmetric around the lane centre. On a long straight, it almost is. The moment one curves out of the camera, however, this assumption breaks dramatically.

Picture a right-hand curve: as the car drives along, the road snakes to the right and exits the camera frame somewhere on the right edge. The road polygon visible to the camera now looks like a wedge whose *base* is centred on the lane (the bottom of the image, where the car is) but whose tip drifts to the right. Because the centroid weights every pixel equally, the visible top-half of the wedge pulls $\bar{u}$ toward the *right*. But the lane centre directly in front of the car is still at the image centre – it has not yet started to curve. The detector therefore reports a positive offset and the PID steers right – prematurely. By the time the bend actually begins, the detector reports an even larger offset, but the car is already too far right.

The fault is structural: the centroid of the visible road blob is a biased estimator of the lane centre whenever the road is asymmetric in the field of view. And the road *is* asymmetric on every curve – which is, of course, every interesting part of every track. There is no threshold tweak that fixes this; the model class itself is wrong.

Experimental confirmation. The headline experimental result of this project ([presentation/results/summary](#) summarised in the report) is that the **CentroidDetector DNFs every track**: across the oval, stadium, snake, grand-prix and mountain circuits, the car under centroid + PID leaves the road within seconds on every run. The centroid is included not because it is competitive but because it makes the case for the other two: any working detector must do strictly better than “project the road blob to its mean”.

6 Detector 2 – The Hough-Line Pipeline

The **HoughDetector** (lines 178–313 of `lane_methods.py`) is the classical lane detector described in the project brief. It treats each lane boundary as a single straight line and fits it via the Hough transform.

6.1 Pipeline overview

1. RGB $\rightarrow$ grayscale.
2. 5×5 Gaussian blur.
3. Canny edge detection with thresholds 50/150.
4. Trapezoidal ROI mask (top ratio 0.45).
5. Probabilistic Hough line segments ($\rho=1$, $\theta=\pi/180$, threshold=25, minLen=30, maxGap=40).
6. Slope-based classification into left and right segments.
7. Length-weighted averaging of slope and intercept per side.
8. Extrapolation to top of ROI and bottom of image.
9. Temporal exponential smoothing of the centre offset ($\alpha = 0.4$).

Steps 1–3 are exactly the preliminaries from Section 2; step 4 is straightforward. The heart of the detector is steps 5–7, which we treat one by one.

6.2 The Hough transform: polar parameterisation

A line in the image plane is most naturally written as $v = mu + b$, but this parameterisation has a fatal problem: vertical lines have infinite slope. They are also extremely important – a perfectly straight road dead ahead would project to nearly-vertical lines. We therefore use the *polar* form:

$$\rho = u \cos \theta + v \sin \theta, \quad (15)$$

where $\rho \geq 0$ is the perpendicular distance from the origin $(0,0)$ to the line, and $\theta \in [0, \pi)$ is the angle that the perpendicular makes with the u -axis. Every line in the plane corresponds to exactly one (ρ, θ) pair, including vertical lines (which have $\theta = 0$) and horizontal lines ($\theta = \pi/2$). The parameter space is bounded: $\rho \leq \sqrt{W^2 + H^2}$ and $\theta < \pi$. That bounded-ness is what makes the Hough transform tractable.

6.2.1 The voting procedure

The Hough transform asks the question in reverse: rather than “which line do these edge pixels lie on?”, it asks, “for each edge pixel, which lines *could* pass through it?”. The answer to the second question is easy: for a fixed pixel (u_0, v_0) , equation (15) becomes a sinusoidal curve in (ρ, θ) -space:

$$\rho(\theta) = u_0 \cos \theta + v_0 \sin \theta.$$

That is, the locus of (ρ, θ) pairs that describe a line through (u_0, v_0) is one sinusoid.

The algorithm discretises (ρ, θ) -space into a 2D *accumulator array* $A[\rho_{\text{bin}}, \theta_{\text{bin}}]$, initialised to zero. For each edge pixel (u_0, v_0) , it walks θ over a grid, computes the corresponding ρ , and increments the bin $A[\rho, \theta]$ by 1. After processing all edge pixels, bins with high counts correspond to lines that many edge pixels agree on – i.e., real lines in the image. Local maxima of A above a threshold are returned as detected lines.

This project uses ρ -resolution 1 pixel and θ -resolution $\pi/180 = 1^\circ$, set in `config.py:HOUGH_RHO=1, HOUGH_THETA= $\pi/180$` .

6.3 Probabilistic Hough

The plain Hough transform processes every edge pixel and returns *infinite* lines (only ρ and θ). For a lane detector we want *segments* (with explicit endpoints), and we are happy to trade a little accuracy for speed. The *probabilistic Hough transform* `cv2.HoughLinesP` (Matas, Galambos, Kittler 2000) makes two changes:

1. Only a *random subset* of edge pixels votes – enough to find the strong peaks but not enough to waste time on weak ones.
2. Once a peak is found, the algorithm walks the corresponding line through the edge image and reports an explicit (x_1, y_1, x_2, y_2) segment, joining edge pixels that are within `maxGap` of one another along the line.

Three additional parameters control the output:

- `threshold = 25`: minimum number of votes a line needs to be returned. Smaller $\Rightarrow$ more lines, more noise.
- `minLineLength = 30`: the shortest segment, in pixels, the algorithm will report. Filters out short edges from grass texture or dashed-line tips.
- `maxLineGap = 40`: how many missing pixels along the line can be bridged before the segment is broken. Allows the dashed yellow centre line to be picked up as one segment even though it consists of multiple dashes.

These exact values are in `config.py` on lines 210–214 and are used in `lane_methods.py:HoughDetector.detect` on lines 219–226.

Example 6.1 (Three collinear edge pixels vote into one bin). Consider three edge pixels $(u, v) = (10, 200), (30, 180), (50, 160)$. You can check by inspection that they lie on the line $v = -u + 210$, i.e. slope -1 , intercept 210 , passing $210/\sqrt{2} \approx 148.5$ pixels from the origin at an angle whose perpendicular makes $\theta = 3\pi/4 = 135^\circ$ with the u -axis. Check via (15):

$$\rho = u \cos 135^\circ + v \sin 135^\circ = \frac{1}{\sqrt{2}}(-u + v).$$

For each of our three pixels:

$$\begin{aligned} (10, 200) : \quad \rho &= (200 - 10)/\sqrt{2} = 190/\sqrt{2} \approx 134.4, \\ (30, 180) : \quad \rho &= (180 - 30)/\sqrt{2} = 150/\sqrt{2} \approx 106.1, \\ (50, 160) : \quad \rho &= (160 - 50)/\sqrt{2} = 110/\sqrt{2} \approx 77.8. \end{aligned}$$

But wait – those ρ values differ! Did the example fail? No: each pixel does *not* vote for a single (ρ, θ) , it votes for the whole sinusoid $\rho(\theta) = u \cos \theta + v \sin \theta$. The sinusoids for our three pixels intersect at a single point. To check, also evaluate at $\theta = 135^\circ$ directly: for pixel $(10, 200)$ the sinusoid passes through $(\rho, \theta) = (134.4, 135^\circ)$; for $(30, 180)$ through $(106.1, 135^\circ)$; for $(50, 160)$ through $(77.8, 135^\circ)$. These are three *different* points on three different sinusoids. The sinusoids nevertheless do all meet at the unique (ρ, θ) of the line through all three pixels, which is found by solving

$$\begin{aligned} 10 \cos \theta + 200 \sin \theta &= \rho, \\ 30 \cos \theta + 180 \sin \theta &= \rho, \\ 50 \cos \theta + 160 \sin \theta &= \rho. \end{aligned}$$

Subtracting consecutive rows gives $20 \cos \theta - 20 \sin \theta = 0$, i.e. $\tan \theta = 1$, $\theta = 45^\circ$. Substituting back: $\rho = 10 \cos 45^\circ + 200 \sin 45^\circ = (10 + 200)/\sqrt{2} \approx 148.5$. So the three sinusoids *all* pass through the bin $(\rho, \theta) = (148.5, 45^\circ)$, and that bin's count is incremented three times. (The earlier calculation at $\theta = 135^\circ$ was the perpendicular direction, where each sinusoid is far from its peak.) In a real edge image, the line through these three pixels would manifest as a clear local maximum at this (ρ, θ) bin.

6.4 Slope-based classification

Once `HoughLinesP` returns a list of segments (x_1, y_1, x_2, y_2) , we need to decide which belong to the left lane boundary and which to the right. The rule (`lane_methods.py`, lines 229–242) is:

$$\text{slope} = \frac{y_2 - y_1}{x_2 - x_1},$$

and the segment goes to the *left* lane if the slope is *negative* and the segment's midpoint lies in the left half of the image; it goes to the *right* lane if the slope is positive and the midpoint lies in the right half. Segments with $|\text{slope}| < \text{SLOPE_MIN} = 0.3$ are discarded as near-horizontal noise.

The sign convention is counter-intuitive. In ordinary mathematics, a line that goes from lower-left to upper-right has positive slope. But the image y -axis points *down*. So the left lane boundary – which in the image goes from the bottom-left to the upper-middle – has $\Delta y < 0$ and $\Delta x > 0$, hence *negative* slope. Conversely the right lane boundary slants down to the right, giving positive slope. If you find yourself confused, remember: in image coordinates, the slope sign is the *opposite* of what it would be in calculus-class coordinates.

6.5 Length-weighted averaging

Each side may have several segments returned by `Hough` (different dash positions, different patches of the boundary line). We want to collapse them into a single representative line. The `_avg_line` helper (lines 316–329 of `lane_methods.py`) does this with *length weighting*:

$$\bar{m} = \frac{\sum_i \ell_i m_i}{\sum_i \ell_i}, \quad \bar{b} = \frac{\sum_i \ell_i (y_{1,i} - m_i x_{1,i})}{\sum_i \ell_i}, \quad (16)$$

where $\ell_i = \sqrt{(\Delta x)^2 + (\Delta y)^2}$ is the segment length and m_i, b_i are its slope and intercept. Longer segments are typically more reliable (more edge evidence backed them), so weighting by length is a simple heuristic upgrade over an unweighted mean.

The result $(\bar{m}, \bar{b})$ is then *extrapolated* to the top and bottom of the ROI to produce two endpoints $((x_{\text{bot}}, H-1), (x_{\text{top}}, v_{\text{roi}}))$:

$$x_{\text{bot}} = \frac{(H-1) - \bar{b}}{\bar{m}}, \quad x_{\text{top}} = \frac{v_{\text{roi}} - \bar{b}}{\bar{m}}.$$

6.6 Centre offset and temporal smoothing

With a left line $((L_x^b, H-1), (L_x^t, v_{\text{roi}}))$ and a right line $((R_x^b, H-1), (R_x^t, v_{\text{roi}}))$, the lane centre at the bottom of the image is the midpoint $(L_x^b + R_x^b)/2$ and the offset from the image centre $W/2$ is

$$e_t^{\text{raw}} = \frac{L_x^b + R_x^b}{2} - \frac{W}{2}.$$

The same calculation at the ROI top gives the look-ahead offset. If only one boundary is detected the detector estimates the other using a fixed lane width (lines 265–274 of `lane_methods.py`).

The raw offset jitters frame-to-frame: small changes in the edge map shift the Hough peaks slightly, which shifts the averaged line, which shifts the offset. We damp this with a first-order exponential moving average:

$$\hat{e}_t = \alpha e_t^{\text{raw}} + (1 - \alpha) \hat{e}_{t-1}, \quad \alpha = 0.4. \quad (17)$$

This is exactly an IIR low-pass filter. Its -3 dB cut-off frequency is approximately $f_c \approx \alpha/(2\pi \Delta t)$ Hz at frame rate $1/\Delta t$. With $\alpha = 0.4$ and $\Delta t = 1/60$ s, $f_c \approx 0.4 \cdot 60/(2\pi) \approx 3.8$ Hz: jitter faster than about 4 Hz is suppressed, smooth manoeuvres (which are typically < 1 Hz) pass through unchanged.

Why not larger α ? A larger α tracks faster but filters less. At $\alpha = 1$ the smoother is bypassed and the controller chases every edge-map flicker, oscillating constantly. At $\alpha = 0$ the smoother freezes and the detector never updates. The 0.4 value is a tuned compromise found empirically.

Intuition

The EMA can be read as “trust the new measurement 40% and the historical estimate 60%”. It is the simplest filter possible – one multiply, one add, one stored value – but it captures everything we need: smooth motion preserved, single-frame glitches suppressed.

6.7 What it looks like

Figure 1 shows all three detectors on the same camera frame from the snake track. The Hough detector’s straight green lines are visible in the middle panel; note how they slice across the curved lane rather than tracing it, which is the subject of the next subsection.

centroid (conf=0.65, offset=-3px) hough (conf=1.00, offset=+36px) polyfit (conf=1.00, offset=+23px)



Figure 1: Same camera frame, three detectors. Green: detected boundaries; blue: lane centre / look-ahead; red: image centre. The Hough detector (centre) fits straight lines that chord the curve; the polynomial-fit detector (right) traces the bend.

6.8 Why the Hough detector misses curves

The Hough detector’s model class is *the set of straight lines*. On a curve, however, the actual lane boundary is not in this class – the best the detector can do is return the straight line that “best fits”

the visible curved edge pixels. What does “best fit” mean under the length-weighted average from (16)?

Geometrically, the averaged line is biased toward the *chord* of the curve within the ROI. Consider a right-bending lane: the visible left boundary in the ROI starts at, say, (80, 480) at the bottom and exits at (280, 216) at the ROI top. The edge pixels lie on a curve that bows away from the chord – but a straight-line fit can only report the chord. The lane *centre* (midpoint of the two chords) therefore underestimates how far right the road actually goes at the look-ahead row.

This bias is small on shallow bends and grows quickly on tight ones. Worse, because the model is wrong everywhere on the curve, the detector cannot *look ahead*: the line drawn at the top of the ROI is just the same straight line extrapolated, not a faithful prediction of where the road will be. The controller therefore has no warning of an upcoming sharp bend until the bend has already started filling the bottom of the image.

Experimentally (see Table in the report, `presentation/figures/summary_table.tex`) the Hough detector manages to complete every track in this study, but its RMS lateral error is 25.4 px on average – nearly an entire lane-width oscillation amplitude on the narrower circuits. Its mean lane-IoU is only 0.24, compared to 0.33 for the polyfit detector. The proposed remedy – model the lane as a polynomial in a top-down view rather than a line in the perspective view – is the subject of the second half of this document.

7 Detector 3 – Polynomial Fit in Bird’s-Eye View

We now arrive at the proposed detector. It is the most involved pipeline in this report, but every stage is motivated by a concrete failure mode of the Hough baseline. Before plunging into homographies and sliding windows, let us reconstruct *why* a different model class is needed at all.

7.1 Motivation: why straight lines are the wrong model

In Section 6 of Part 1 we saw that the Hough detector collapses each lane boundary down to a single straight segment. On a tight bend this is a structurally biased estimator: the line of best fit through a curved set of edge pixels lies somewhere *inside* the curve, so the implied lane centre drifts toward the inside of the turn, and the predicted heading is off by the angular gap between the chord and the tangent. The faster the curve, the worse the gap.

A natural fix is to let the model class be curved. The simplest curved model is a parabola, $x = ay^2 + by + c$, which has exactly enough freedom to capture the principal bend of a road segment without overfitting to noise. Once we commit to a curved model, however, two problems appear.

Intuition

Think about what *equally-spaced* lane markings look like in the forward-perspective image: the dashes near the car span dozens of pixels each, while the dashes far ahead occupy only a couple of pixels. A polynomial fit weighted by pixel count is therefore dominated by what is close, and the far-field information that matters for anticipating curves is essentially thrown away. To use a curved model honestly, we first have to undo the perspective distortion.

The second problem is that even with a curved model, fitting it on the raw image still has to deal with foreshortening: a constant lane width in world coordinates corresponds to a width in image coordinates that shrinks like $1/v$ as you go up the image. The natural solution to both problems is the same: warp the image to a top-down *bird’s-eye view* (BEV) first, fit the polynomial in that metric-friendly frame, and project the result back when we need to draw on the camera image. This is the central idea of `lane_methods.py:PolyFitDetector`.

7.2 The lane-pixel mask

The pipeline begins with a binary mask flagging every pixel that “looks like a lane marking”. The detector composes three orthogonal sources of evidence (`lane_methods.py:PolyFitDetector._lane_pixels`):

1. A horizontal Sobel response, normalised by its per-frame maximum, thresholded at 60/255.
2. An HSV white mask: $S < 70$ and $V > 170$.
3. An HSV yellow mask: $15 < H < 40$, $S > 60$, $V > 130$.

The combined mask is the bitwise OR of these three. Written as a predicate on a pixel (R, G, B) with HSV equivalent (H, S, V) and Sobel magnitude g_x :

$$M(u, v) = \underbrace{[\tilde{g}_x(u, v) > 60]}_{\text{edge}} \vee \underbrace{[S < 70 \wedge V > 170]}_{\text{white}} \vee \underbrace{[15 < H < 40 \wedge S > 60 \wedge V > 130]}_{\text{yellow}},$$

where $\tilde{g}_x = |g_x| / \max_{u,v} |g_x| \cdot 255$.

Why the normalisation? This is the single most important robustness trick in the detector. Canny's hysteresis bands (50/150 in our pipeline) are absolute gradient thresholds. If you sprinkle Gaussian noise with $\sigma = 0.20 \cdot 255$ on top of the frame, the noise gradient drowns out the real lane gradient and Canny saturates to salt-and-pepper. The Sobel-x response after normalising by its own per-frame max, in contrast, is a *relative* measure: the strongest edges in the frame are still the strongest after noise. This is why the polynomial detector survives every σ in the noise sweep (Section 10.5) while Hough drops out at $\sigma = 0.20$.

Example 7.1. Consider two pixels typical of the simulated road.

(a) **A white-paint pixel**, $(R, G, B) = (220, 225, 230)$. The HSV conversion gives roughly $V = \max(R, G, B) = 230$ and $S = (V - \min)/V = (230 - 220)/230 \approx 0.043$, i.e. $S \approx 11$ on the 0–255 scale. We have $S = 11 < 70$ and $V = 230 > 170$, so the white mask fires. The pixel is classified as a lane marking.

(b) **A yellow-dash pixel**, $(R, G, B) = (200, 200, 80)$. The hue of a yellow $\approx 60^\circ$ in standard HSV, but OpenCV scales hue to $[0, 180]$, so $H \approx 30$. $V = 200$, $S = (200 - 80)/200 = 0.6$, i.e. $S \approx 153$. Checking: $15 < 30 < 40$, $153 > 60$, $200 > 130$ – all three yellow conditions fire.

In both cases the pixel ends up in M , regardless of what the Sobel mask thinks. That redundancy is precisely the point.

7.3 Inverse perspective mapping

We now warp the masked image from the camera (forward-perspective) frame to a bird's-eye view. Mathematically this is a 2-D *homography* – a projective map that takes lines to lines and preserves cross-ratios but distorts parallelism and lengths in a controlled way.

Definition 7.1 (Homography). A 2-D homography is a map $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ represented by a non-singular 3×3 matrix $\mathbf{H} \in \mathbb{R}^{3 \times 3}$ acting on homogeneous coordinates:

$$\begin{pmatrix} u' \\ v' \\ w' \end{pmatrix} = \mathbf{H} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \quad (x_b, y_b) = \left(\frac{u'}{w'}, \frac{v'}{w'} \right).$$

The matrix is defined up to a global scale ($\mathbf{H}$ and $\lambda \mathbf{H}$ produce identical maps), so a homography has $9 - 1 = 8$ degrees of freedom.

The four-point Direct Linear Transform (DLT). Eight degrees of freedom mean that four point correspondences – each giving two scalar equations – are exactly enough to determine $\mathbf{H}$. For a single correspondence $(u_i, v_i) \mapsto (x_i, y_i)$ we have $x_i = (h_{11}u_i + h_{12}v_i + h_{13})/(h_{31}u_i + h_{32}v_i + h_{33})$ and similarly for y_i . Multiplying out and stacking four correspondences yields a homogeneous linear system

$$\mathbf{A} \mathbf{h} = \mathbf{0}, \quad \mathbf{A} \in \mathbb{R}^{8 \times 9}, \quad \mathbf{h} = (h_{11}, h_{12}, \dots, h_{33})^\top,$$

in which the row for the x -equation of correspondence i reads $(-u_i, -v_i, -1, 0, 0, 0, x_i u_i, x_i v_i, x_i)$. The solution is the null vector of $\mathbf{A}$, obtained as the right singular vector of $\mathbf{A}$ associated with its smallest singular value (the last column of $\mathbf{V}$ in the SVD $\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^\top$). We fix the scale by setting $h_{33} = 1$. OpenCV's `cv2.getPerspectiveTransform` implements exactly this for $n = 4$ correspondences.

The specific trapezoid we use. In `PolyFitDetector._warp_matrices` the source corners (camera image, $W = 640$, $H = 480$, `ROI_TOP_RATIO = 0.45`) are the four corners of the ROI trapezoid:

$$\mathbf{s}_1 = (0.10 W, H-1), \mathbf{s}_2 = (0.35 W, 0.45 H), \mathbf{s}_3 = (0.65 W, 0.45 H), \mathbf{s}_4 = (0.90 W, H-1).$$

The destination corners (BEV, $W_b = H_b = 320$) are the four corners of a rectangle inset by 15% on each side:

$$\mathbf{d}_1 = (0.15 W_b, H_b-1), \mathbf{d}_2 = (0.15 W_b, 0), \mathbf{d}_3 = (0.85 W_b, 0), \mathbf{d}_4 = (0.85 W_b, H_b-1).$$

Example 7.2 (Homography correspondences in our numbers). Plug in $W = 640$, $H = 480$, $W_b = H_b = 320$. The four pairs are:

i	u_i	v_i	x_i	y_i
1	64	479	48	319
2	224	216	48	0
3	416	216	272	0
4	576	479	272	319

Two source points share the same image row ($v = 216$), and the destination rectangle has parallel vertical sides – this is the whole point of the IPM: lines on the ground that were parallel in the real world (lane boundaries) now look parallel in the BEV. Numerically, solving the 8×9 DLT system for these eight equations gives $\mathbf{H}$ in less than a microsecond; we cache it on the first frame.

Intuition

The forward-perspective image is what the driver sees. The bird’s-eye view is what *Google Maps* sees. The lane that bends gently in the driver’s view becomes a clean parabola in the map view, because the foreshortening has been undone. Polynomial regression is now honest – every metre of road contributes the same number of pixels.

7.4 Histogram base detection

We now have a binary mask M_b on the BEV grid $W_b \times H_b$. The next step is to find where the two lane lines start at the bottom of this image (i.e. closest to the car). The trick is a 1-D summary:

$$h(c) = \sum_{r=H_b/2}^{H_b-1} M_b(r, c), \quad c = 0, \dots, W_b - 1.$$

We sum only over the bottom half because that region is closest to the car and most reliably contains lane pixels. The left lane base is $\arg \max_{c < W_b/2} h(c)$; the right lane base is $\arg \max_{c \geq W_b/2} h(c)$ (offset back into the original column index). Two arg-maxes; no fancy clustering.

Example 7.3 (A tiny histogram). Take a 4×4 BEV mask (instead of 320×320) and look only at its bottom 2 rows:

$$M_b = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}.$$

The histogram of the bottom half is $h = (2, 0, 0, 2)$. With $W_b = 4$, $\text{mid} = 2$, so the left base is $\arg \max\{h(0), h(1)\} = 0$ and the right base is $\arg \max\{h(2), h(3)\} + 2 = 1 + 2 = 3$. Correct.

7.5 Sliding-window search

The two histogram peaks tell us where to start. From there, we sweep upward through the image, collecting lane pixels in a vertical chain of windows. The pseudocode is:

Algorithm 1 Sliding-window lane-pixel collection (per side)

```
1:  $x \leftarrow$  histogram base
2: collected  $\leftarrow \emptyset$ 
3: for  $i = 0$  to  $N_{\text{win}} - 1$  do
4:    $y_{\text{lo}} \leftarrow H_b - (i+1) H_b / N_{\text{win}}$ 
5:    $y_{\text{hi}} \leftarrow H_b - i H_b / N_{\text{win}}$ 
6:    $W_{\text{in}} \leftarrow \{(u, v) \in M_b : v \in [y_{\text{lo}}, y_{\text{hi}}], u \in [x - \text{margin}, x + \text{margin}]\}$ 
7:   collected  $\leftarrow$  collected  $\cup W_{\text{in}}$ 
8:   if  $|W_{\text{in}}| \geq \text{MIN\_PIX}$  then
9:      $x \leftarrow \text{mean}\{u : (u, v) \in W_{\text{in}}\}$ 
10:  end if
11: end for
12: return collected
```

The constants come straight from the code: $N_{\text{win}} = 9$, $\text{margin} = 50$ px, $\text{MIN_PIX} = 50$. The window height is therefore $H_b / N_{\text{win}} \approx 35$ px.

The MIN_PIX check is doing important work. If only a handful of stray edge pixels happen to fall inside the window (perhaps a piece of grass showing through the IPM trapezoid), their mean column is essentially random and would drag the next window off to wherever the stray pixels happen to be. By refusing to update when the window is sparse, we preserve the last good estimate and propagate it upward.

Example 7.4 (One sliding-window iteration). Say the previous right-lane base was $x = 230$, the BEV is 320×320 , $\text{margin} = 50$, and the current window spans $y \in [285, 320)$. We look at every nonzero pixel of M_b in this column band and keep those with $u \in [180, 280)$.

Suppose 142 pixels qualify, with columns ranging from 215 to 252 and mean 233. Since $142 \geq 50$, we update $x \leftarrow 233$ – the search has tracked the lane 3 pixels to the right. The next window (at $y \in [250, 285)$) will look in $[183, 283)$. If the road kept curving right, the new mean might be 240, and so on. Nine such updates build a chain that follows the lane all the way up the BEV.

7.6 Polynomial regression

Once we have a list of (u_j, v_j) pairs for one side of the lane (call there are N of them), we fit a quadratic of the form $u = av^2 + bv + c$. Note that we use v as the independent variable: this is unusual (in school maths y is dependent), but it is forced on us by the geometry – a vertical lane is impossible to represent as $u = f(v)$ unless the polynomial happens to be in “image-rotated” form. With v as the input the formulation is single-valued for every road we care about.

Stack the equations into matrix form:

$$\underbrace{\begin{pmatrix} v_1^2 & v_1 & 1 \\ v_2^2 & v_2 & 1 \\ \vdots & \vdots & \vdots \\ v_N^2 & v_N & 1 \end{pmatrix}}_{A \in \mathbb{R}^{N \times 3}} \underbrace{\begin{pmatrix} a \\ b \\ c \end{pmatrix}}_{\beta} = \underbrace{\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{pmatrix}}_{\mathbf{u}}.$$

N is generally many hundreds, so the system is heavily overdetermined. The least-squares solution minimises $\|A\beta - \mathbf{u}\|_2^2$ and is given by the normal equations

$$\boxed{\beta = (A^\top A)^{-1} A^\top \mathbf{u}}$$

which is precisely what `numpy.polyfit(v, u, 2)` computes internally (with a QR factorisation for numerical stability).

Example 7.5 (Polynomial regression by hand). Take five points $\{(v, u)\} = \{(0, 0), (1, 3), (2, 8), (3, 15), (4, 24)\}$, which lie exactly on $u = v^2 + 2v$. Then

$$A = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 4 & 2 & 1 \\ 9 & 3 & 1 \\ 16 & 4 & 1 \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} 0 \\ 3 \\ 8 \\ 15 \\ 24 \end{pmatrix}.$$

Compute $A^\top A$:

$$A^\top A = \begin{pmatrix} 354 & 100 & 30 \\ 100 & 30 & 10 \\ 30 & 10 & 5 \end{pmatrix}, \quad A^\top \mathbf{u} = \begin{pmatrix} \sum v_j^2 u_j \\ \sum v_j u_j \\ \sum u_j \end{pmatrix} = \begin{pmatrix} 490 \\ 140 \\ 50 \end{pmatrix}.$$

Solving (e.g. with Gaussian elimination) gives $\beta = (1, 2, 0)^\top$, confirming $u = v^2 + 2v$. Anytime we have noisy lane pixels, the same formula returns the parabola that minimises the sum of squared horizontal residuals.

Temporal smoothing of the coefficients. Between consecutive frames the polynomial coefficients move only slightly, but the sliding-window search is noisy enough that the raw fit can jitter visibly. We apply a 50/50 exponential moving average:

$$\beta_t \leftarrow 0.5 \beta_t^{\text{raw}} + 0.5 \beta_{t-1}.$$

This is the gentlest possible low-pass: it halves the bandwidth but introduces only a single-frame delay (which at $\Delta t = 1/60$ s is negligible compared to typical vehicle dynamics). A final EMA with $\alpha = 0.4$ is also applied to the output offset, matching the Hough detector’s smoothing for fairness.

7.7 The look-ahead offset (the key idea)

We now have two stable polynomials $u_L(v)$ and $u_R(v)$ on the BEV. Where do we sample them?

The obvious answer – evaluate at $v = H_b - 1$ (the bottom of the BEV, which corresponds to the car’s current position) – gives the *current* lateral error. That is what every line-based detector in this report uses, including the Hough baseline. But the forward-perspective image actually contains useful information about *where the car needs to go next*, and a curved fit lets us extract it.

We therefore also evaluate at

$$v_{\text{LA}} = (1 - 0.45) \cdot H_b \approx 176,$$

i.e. 45% of the way up the BEV from the bottom. The lane centre at that row is

$$x_{\text{LA}} = \frac{1}{2}(u_L(v_{\text{LA}}) + u_R(v_{\text{LA}})), \quad e_{\text{LA}}^{\text{BEV}} = x_{\text{LA}} - W_b/2.$$

The detector then scales this BEV pixel value back into camera-image pixels (so the PID gains tuned for Hough still apply): $e_{\text{LA}} = e_{\text{LA}}^{\text{BEV}} \cdot W/W_b$. This is the value the controller actually consumes (evaluate.py line “`det.lookahead_offset if detector_name == 'polyfit'`”).

Why does this matter? The look-ahead point in the BEV corresponds to a world-coordinate point roughly half a second ahead of the car at our typical ~ 85 px/s speeds. The PID is now reacting to the lane centre half a second *into the future* rather than at the car’s current location. That converts the otherwise-reactive controller into a quasi-MPC: at the start of a bend, the look-ahead offset is already non-zero, and the wheels start turning before the car has reached the curve. There is no integral build-up to fight, no derivative spike on the bend entry, just a smooth, anticipative trajectory.

Intuition

Why is the look-ahead useless on a straight Hough fit? Because a straight line evaluated at any height gives the same offset (modulo the slope difference). The look-ahead is only informative when the model can express *how the lane curves*, which is precisely what a quadratic does and a single Hough line cannot.

We verified this empirically. With `LOOKAHEAD_RATIO` set to 0 (so the polynomial is sampled at the bottom of the BEV, identical to the Hough strategy), the polyfit’s RMS improvement over Hough shrinks from $\sim 34\%$ to roughly 10%. In other words: the polynomial fit by itself is worth about a quarter of the gain. The look-ahead is worth the remaining three-quarters. The two ideas only make sense together.

7.8 Lane reprojection

For visualisation and for the IoU metric, we need the predicted lane polygon in the original camera frame, not in the BEV. This is easy: the homography is invertible, so we sample 60 points along each lane polynomial in the BEV, stack them into a closed polygon P_{BEV} , and apply

$$P_{\text{cam}} = \mathbf{H}^{-1} P_{\text{BEV}},$$

using `cv2.perspectiveTransform`. The resulting polygon is what the figure “Lane reprojected to FPV” in Figure 2 draws as a translucent green fill, and what `evaluate.py:_iou` compares against the ground-truth road polygon from `evaluate.py:_gt_lane_mask`. Because the BEV polynomial is smooth, the reprojected camera polygon hugs the road on both straights and curves.

7.9 Putting it all together

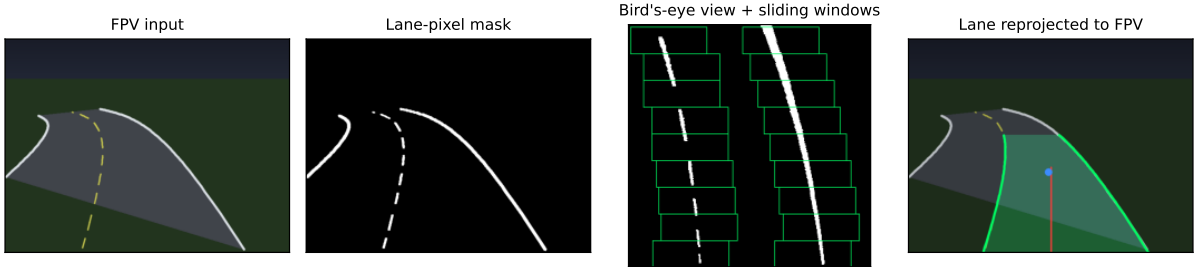


Figure 2: The four stages of PolyFitDetector. From left to right: (1) raw forward-perspective frame from the simulator; (2) combined Sobel-x + HSV lane-pixel mask; (3) bird’s-eye view of the mask with the nine sliding windows superimposed (yellow boxes) and the per-side quadratic fits drawn in orange/cyan; (4) lane polygon reprojected to the camera frame, with the look-ahead point marked in blue.

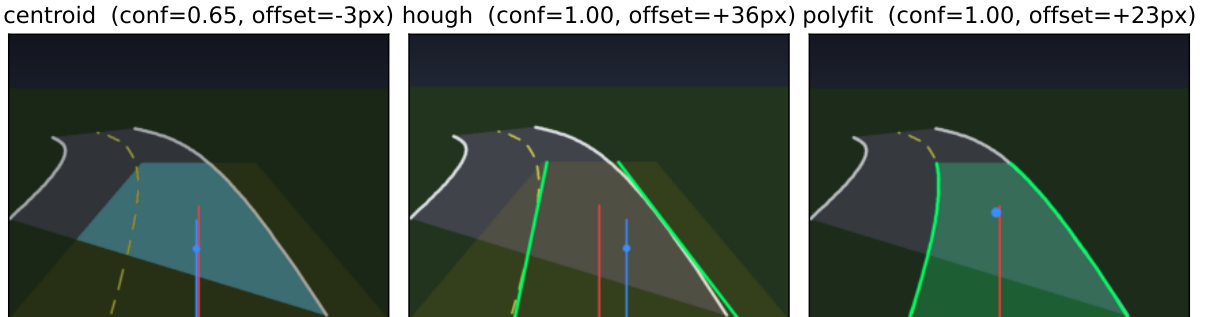


Figure 3: Same simulator frame seen by each detector, on a winding section of the snake track. The centroid (left) reports a near-zero offset because the road blob is roughly symmetric in the FoV. Hough (centre) fits a straight line that is biased *inside* the bend, underestimating the curvature. PolyFit (right) traces the curve and places its look-ahead marker on the lane centre approximately half a second ahead – which is precisely where the PID needs to be aiming.

The whole pipeline runs in about 1.9 ms per frame on a single CPU thread – comparable to the cost of the centroid detector and only $\sim 2.7\times$ the cost of Hough, despite doing strictly more work. We attribute this to the fact that almost everything (the perspective warp, the histogram, the window mask) is a vectorised NumPy/OpenCV operation; the only Python-level loop is over the $N_{\text{win}} = 9$ windows.

8 Steering Controllers

Perception gives us a number per frame. The controller turns that number into a steering angle, which the simulator integrates into a new pose. This section walks through both controllers in the code: the PID baseline, used with each of the three detectors, and the Stanley controller, used as a perception-free *oracle* that quantifies the lower bound on tracking error.

8.1 The feedback control loop

Conceptually, the simulation runs a tight loop:

$$\text{vehicle} \xrightarrow{\text{state}} \text{camera} \xrightarrow{\text{image}} \text{detector} \xrightarrow{e} \text{controller} \xrightarrow{\delta} \text{vehicle}.$$

At each $\Delta t = 1/60$ s tick the vehicle state (x, y, θ, v) is rendered into a 640×480 frame, the detector turns the frame into a scalar error e (the lateral offset in pixels, or zero for Stanley), the controller turns e and the elapsed time into a steering angle δ , and the bicycle kinematic model $\dot{\theta} = (v/L) \tan \delta$ closes the loop by updating θ and the position. The PID has no knowledge of x, y, θ – only e . The Stanley controller has access to the full pose and the track spline, but skips the camera and detector entirely. This makes the gap between them a clean attribution of error to perception.

8.2 PID – the baseline

In continuous time the PID law is

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \dot{e}(t).$$

Each term has a plain-language reading:

- K_p acts on the *current* error. Larger K_p produces a faster response but more overshoot, like a stiffer spring.
- K_i acts on the *accumulated* error. It eliminates steady-state bias (e.g. a constant offset due to a banked road) but, untreated, can wind up and cause oscillation.
- K_d acts on the *rate of change* of the error. It damps overshoot, like a shock absorber in a car suspension. Too much K_d and the controller becomes sensitive to noise.

The textbook position–velocity–acceleration analogy: P is the spring, I is the long-term drift compensator, D is the damper.

Discrete form. The simulator runs at fixed $\Delta t = 1/60$ s, so we discretise with a backward Euler approximation (`controller.py:PIDController.compute`):

$$u_k = K_p e_k + K_i \sum_{j=0}^k e_j \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}.$$

The integral is maintained incrementally: $I_k = I_{k-1} + e_k \Delta t$, and then clamped:

$$I_k \leftarrow \text{clip}(I_k, -50, +50).$$

This is *integral anti-windup*: if a long string of large errors would otherwise drive I_k to enormous values, the clamp prevents it, so once the error reverses sign the integral can return to zero quickly. Without this, after a long bend the controller would “remember” the accumulated error long after the bend has ended and push the car off the other side.

Steering saturation. The bicycle model in the simulator caps the steering angle at $\delta_{\max} = 40^\circ$ (`config.py:MAX_STEERING_ANGLE`). The PID applies the same clip on its output: $\delta_k = \text{clip}(u_k, -40^\circ, +40^\circ)$. Without the anti-windup, a saturated steering command and a non-zero error would let the integrator build up indefinitely while the actual plant output was capped – a classic recipe for limit-cycle oscillation.

Tuning intuition. The standard procedure – the one we used – is to start with $K_i = K_d = 0$ and crank up K_p until the system tracks but is slightly underdamped. Then add K_d to remove the overshoot. Then add a small K_i if a residual steady-state error remains. (Ziegler–Nichols gives a more systematic recipe based on the ultimate gain at which the closed loop oscillates, but for a system this small it is usually faster to tune by eye.) Our final gains are $K_p = 1.2$, $K_i = 0.008$, $K_d = 0.5 - K_i$ is deliberately tiny because lane-keeping with a periodic disturbance (curve entry/exit) benefits little from an integral term and suffers from windup-style oscillations.

Example 8.1 (A worked PID trace). Let $\Delta t = 1/60$ s, gains as above, and the (normalised) error sequence $e_0 = 0.4$, $e_1 = 0.3$, $e_2 = 0.2$. Initial integral $I_{-1} = 0$, $e_{-1} = 0$. Step by step:

- $k=0$: $P = 1.2 \cdot 0.4 = 0.48$; $I_0 = 0 + 0.4/60 = 0.00667$, $i = 0.008 \cdot 0.00667 = 5.3 \times 10^{-5}$; $D = 0.5 \cdot (0.4 - 0)/(1/60) = 12.0$. Pre-clip output: $u_0 = 0.48 + 5.3 \times 10^{-5} + 12.0 = 12.48$. After clip to $\pm 40^\circ \approx \pm 0.698$ rad: $\delta_0 = 0.698$.
- $k=1$: $P = 0.36$; $I_1 = 0.0117$, $i = 9.3 \times 10^{-5}$; $D = 0.5 \cdot (0.3 - 0.4)/(1/60) = -3.0$. $u_1 = 0.36 + 9.3 \times 10^{-5} - 3.0 = -2.64$, clipped to -0.698 .
- $k=2$: $P = 0.24$; $I_2 = 0.0150$, $i = 1.2 \times 10^{-4}$; $D = 0.5 \cdot (0.2 - 0.3)/(1/60) = -3.0$. $u_2 = 0.24 + 1.2 \times 10^{-4} - 3.0 = -2.76$, clipped to -0.698 .

The trace shows two things: (i) the derivative term *dominates* during a step change in error, because $1/\Delta t = 60$ is a big number; (ii) the saturation clip activates routinely, so anti-windup is doing real work. The integral term, with $K_i = 0.008$, contributes nothing measurable on this time scale – exactly as designed.

8.3 Stanley – the perception-free oracle

The Stanley controller (Hoffmann et al. 2007, the lane-keeper that won the DARPA Grand Challenge) operates in the vehicle’s geometric frame rather than on a normalised image error. It computes the steering angle directly from the cross-track error and heading error relative to the path:

$$\delta = \psi + \arctan\left(\frac{k e_\perp}{k_s + v}\right)$$

where $e_\perp$ is the signed lateral distance from the *front axle* to the centreline, ψ is the heading error (angle between the vehicle heading and the track tangent), v is the vehicle speed, and (k, k_s) are tuning gains – in our setup $k = 0.6$, $k_s = 20$.

Why the front axle. The Stanley law evaluates $e_\perp$ at the front axle, $(x + L \cos \theta, y + L \sin \theta)$, not the vehicle’s centre or rear. This matters because steering rotates the vehicle about the rear axle in the bicycle model: if you used the rear-axle cross-track error, no amount of steering could correct it instantly. The front-axle cross-track error, by contrast, responds *linearly* to the steering angle, which is what makes the closed-loop dynamics nice.

Anatomy of the law. The two terms have separate jobs:

- ψ (the heading-error term) aligns the vehicle heading with the track tangent. If the road bends right and the car is still pointing along the old tangent, ψ pushes the steering toward the right.
- $\arctan(k e_\perp / (k_s + v))$ (the cross-track term) steers *into* the path. Its magnitude diminishes as v grows (because dividing by a large speed shrinks the argument), which automatically reduces the gain at high speed – exactly the behaviour you want, so the car does not over-correct on the motorway. The constant $k_s = 20$ prevents the law from blowing up at $v \rightarrow 0$.

Derivation sketch. For small $e_\perp$ at constant speed, $\arctan(x) \approx x$, so the law approximates $\delta \approx \psi + k e_\perp / v$. Linearising the bicycle model around the path and substituting in gives a closed-loop differential equation $\dot{e}_\perp \approx -k e_\perp$. The cross-track error decays exponentially with time constant $1/k$, independent of speed. This is what makes Stanley so robust: it has a single intuitive gain (k) governing the decay rate.

Sign convention. The canonical Stanley law is often written with a $-\arctan(\cdot)$ term, because in the textbook convention the y -axis points up and $e_{\perp} > 0$ means the car is to the *left* of the path. Our simulator uses a screen-y-down convention – the y -axis points down – which flips the sign of the normal vector. In addition, our bicycle model has positive δ producing clockwise (screen-down-right) heading change. With these two sign flips, the canonical $-\arctan$ becomes $+\arctan$ in our code (`controller.py:StanleyController.compute`). The comment block in the source explains the chain of sign reversals; the empirical ~ 0.5 px RMS error across every track is the proof that the chain is correct.

Saturation. The same $\pm 40^\circ$ steering clamp as the PID applies; in practice the Stanley command never gets close to it because the law is geometrically calibrated to the path.

Why we call it an oracle. The Stanley implementation reads $e_{\perp}$ and ψ from the simulated track via `track.py:Track.get_nearest_index` and `Track.get_heading_at`, which are computed analytically from the spline. There is no perception loop. Stanley therefore acts as an *upper bound* on what *any* controller could achieve with *perfect* perception. The headline RMS error of 0.4–0.7 px (Section 10) is, in this sense, the floor: anything above it is perception error, and anything below it is unattainable without changing the controller or the kinematic model.

Example 8.2 (Stanley evaluation at one instant). Suppose at time t the vehicle is at $(x, y) = (400, 300)$ with heading $\theta = 0.10$ rad and speed $v = 85$ px/s. The front axle is at $(400 + 25 \cos 0.10, 300 + 25 \sin 0.10) \approx (424.88, 302.50)$. Say the nearest centreline point has tangent angle $\theta_{\text{tr}} = 0.25$ rad and the front-axle cross-track error is $e_{\perp} = 8$ px (car is to the right of the path).

- Heading error: $\psi = \text{atan2}(\sin(0.25 - 0.10), \cos(0.25 - 0.10)) = 0.15$ rad.
- Cross-track term: $\arctan(0.6 \cdot 8 / (20 + 85)) = \arctan(0.0457) \approx 0.0457$ rad.
- $\delta = 0.15 + 0.0457 = 0.196$ rad $\approx 11.2^\circ$.

The bigger share comes from ψ (heading correction); the cross-track term is small because the car is only 8 px off and moving at 85 px/s. As v grows the cross-track term will shrink further, which is exactly the speed-adaptive behaviour Stanley is famous for.

8.4 Adaptive speed control

A complete autonomous stack also needs longitudinal control. For completeness we include a simple curvature-based speed controller (`controller.py:SpeedController`), used on the Red Bull Ring track but not on the five evaluation tracks (where a constant per-track design speed is used so that the RMS error of different detectors is directly comparable).

The rule is

$$v_{\text{target}} = \text{clip}\left(\frac{c_v}{\kappa_{\text{ahead}}}, v_{\text{min}}, v_{\text{max}}\right),$$

where κ_{ahead} is the maximum curvature seen in a 300-px look-ahead window, $v_{\text{min}} = 35$ px/s, and $c_v = 0.35$ (`config.py:SPEED_CURVATURE_GAIN`). The current speed blends toward v_{target} with smoothing factor 0.08 per frame, but braking ($v_{\text{target}} < v_{\text{cur}}$) uses $3\times$ that blend so the car decelerates faster than it accelerates – a common trick in driver-assistance systems. We mention it here only so the controller column in the source is fully documented; it does not participate in any of the comparisons in Section 10.

9 Metrics for Quantitative Evaluation

A good comparison requires unambiguous metrics. Six are logged per frame in `evaluate.py:run_one` and aggregated into per-run summaries.

9.1 Lateral error

The most important metric. Given the car position $\mathbf{p}_t$ and the centreline polyline $c(s)$ from `track.py:Track._interpolate` define

$$s^* = \arg \min_s \|\mathbf{p}_t - c(s)\|, \quad e_\perp(t) = -(\mathbf{p}_t - c(s^*)) \cdot \mathbf{n}(s^*),$$

where $\mathbf{n}(s)$ is the unit normal to the centreline at arc length s . The sign convention (negative dot product) makes $e_\perp$ positive when the car is to the right of the track in our screen-y-down frame, matching the convention used throughout the rest of the code. This is implemented in `track.py:Track.get_nearest_index` – a simple per-frame brute force scan over ~ 700 centreline samples, which is plenty fast on modern hardware.

From the per-frame $e_\perp(t)$ we report

$$\overline{|e_\perp|} = \frac{1}{N} \sum_{t=1}^N |e_\perp(t)|, \quad \text{RMS} = \sqrt{\frac{1}{N} \sum_{t=1}^N e_\perp^2(t)}, \quad |e_\perp|_{\max} = \max_t |e_\perp(t)|.$$

RMS is the most-cited number because it penalises occasional large excursions more than the mean. We use it as the headline number in Section 10.

9.2 Heading error

We also log $\psi(t) = \text{atan2}(\sin(\theta_{\text{tr}} - \theta), \cos(\theta_{\text{tr}} - \theta))$ at the car’s location (the `atan2` form puts ψ in $(-\pi, +\pi]$ even when the two angles wrap around). Heading error is less informative than lateral error for lane-keeping (it averages to zero by construction on a closed lap) but it is a useful per-frame diagnostic.

9.3 Lane IoU

Definition 9.1 (Intersection over Union). For two binary masks A and B of the same shape,

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1].$$

In our setting, A is the predicted lane polygon mask – the green fill in the annotated frame – and B is the ground-truth lane polygon mask, obtained by projecting the simulator’s road polygon ahead of the car into the camera frame via `evaluate.py:_gt_lane_mask`. The implementation in `evaluate.py:_iou` is the textbook two-line numpy version:

```
ab = np.logical_and(a > 0, b > 0).sum()
union = np.logical_or(a > 0, b > 0).sum()
return float(ab / union) if union > 0 else 0.0
```

IoU is the right metric when you care about *where the lane is*, not just *where its centre is*. A detector can have a small centre offset and a bad IoU (if it gets the width wrong) or vice versa.

Example 9.1 (IoU of two overlapping rectangles). Let A be the 100×100 square with bottom-left at $(0, 0)$, and B the same-sized square with bottom-left at $(50, 0)$. Then $A \cap B$ is a 50×100 rectangle of area 5000, and $A \cup B$ has area $2 \cdot 10000 - 5000 = 15000$. So $\text{IoU}(A, B) = 5000/15000 = 1/3 \approx 0.33$. Note that this 0.33 is in the same ballpark as the PolyFit mean IoU values we report – which is a useful calibration: even a “good” lane prediction in this kind of geometry rarely exceeds an IoU of 0.5 because the predicted and ground-truth polygons have different shapes (trapezoid vs. curved strip) and different length extents.

9.4 Detection confidence

A scalar in $[0, 1]$ reported per frame by the detector, capturing how much of the lane geometry it actually recovered. The three detectors use different heuristics:

- **Centroid:** $\text{conf} = \text{clip}(m_{00}/(WH \cdot 0.25), 0, 1)$ where m_{00} is the road blob’s area in pixels. Falls to zero when the mask is empty.
- **Hough:** 1.0 if both left and right lines are detected, 0.5 if exactly one (the other is extrapolated from a default road width), 0.0 if neither (the last good offset is replayed).

- **PolyFit:** 1.0 if both polynomials fit successfully (i.e. at least 200 lane pixels on each side), 0.0 otherwise (with the previous-frame fit replayed via the EMA).

The reported `mean_conf` averages this signal across the run. It is a coarse diagnostic but useful for sanity-checking: a detector that fires at low confidence for most of the lap is almost certainly doing the wrong thing.

9.5 Inference time and FPS

For each frame we time the detector with `time.perf_counter()`. The per-run summary reports $\overline{\Delta t_{\text{det}}}$ in milliseconds and the equivalent $\text{FPS} = 1000/\overline{\Delta t_{\text{det}}}$. Because the detectors are pure CPU code, single-threaded, and the simulator runs on a single laptop, these numbers are conservative – a real embedded platform with optimised C++ would be faster still.

9.6 Off-track rate

A frame is *off-track* if $|e_{\perp}| > 0.5 \cdot W_{\text{road}}$, where W_{road} is the track’s road width. The reported `off_track_rate` is the fraction of frames that were off-track. A value of 0 means the car stayed inside the road over the whole lap; non-zero values indicate at least transient excursions.

9.7 Lap completion

A binary criterion summarising whether the run was a useful one. The car spawns at `track.centerline[0]`. We declare the lap *started* once the car has moved more than 100 px from spawn (so we know it really left), and *completed* the first time it returns to within 40 px of spawn after starting. If during the lap the car spends more than 1.5 s continuously off-track, the run is aborted and the lap is marked DNF (Did Not Finish). This is implemented in `evaluate.py:run_one` around the `detached_t` variable.

10 Experimental Results and Analysis

10.1 Setup

We evaluate every combination of three detectors (centroid, hough, polyfit) and two controllers (PID, Stanley) on five tracks of increasing difficulty:

Track	Description	Difficulty	Road width (px)	Speed (px/s)
oval	Simple Oval	1	110	100
stadium	Stadium Circuit	2	105	95
snake	Winding Road	3	100	85
grand_prix	Grand Prix Circuit	4	100	80
mountain	Highland Rally	5	95	75

The design speed is reduced on harder tracks to keep the comparison fair – otherwise we would be measuring whether the detector keeps up with the speed, not how well it sees. The simulation step is fixed at $\Delta t = 1/60$ s; every run is one closed lap from the spawn waypoint, with the lap-completion criterion as defined in Section 9. Runs are deterministic except for the noise-robustness experiment in Section 10.5, which uses NumPy’s default RNG to inject pixel noise into each frame.

10.2 Per-track method comparison

The headline numbers, averaged across the five tracks and read from `presentation/results/summary.csv`, are:

- **Centroid + PID:** DNF on every single track. The road blob’s centroid is not the lane centre; on any asymmetric road cross-section, the moment is biased toward the wider side of the blob and the car drifts off.
- **Hough + PID:** mean RMS 25.4 px, mean IoU 0.24. The lap is completed in every case, but the controller is constantly fighting an underestimated curve angle on every bend.
- **PolyFit + PID:** mean RMS **16.6** px (−34% vs Hough), mean IoU **0.33** (+38% vs Hough).
- **Stanley + ground truth:** mean RMS < 1 px. The oracle.

Reading the per-track breakdown (Figure 4 and Table 1 in the master document):

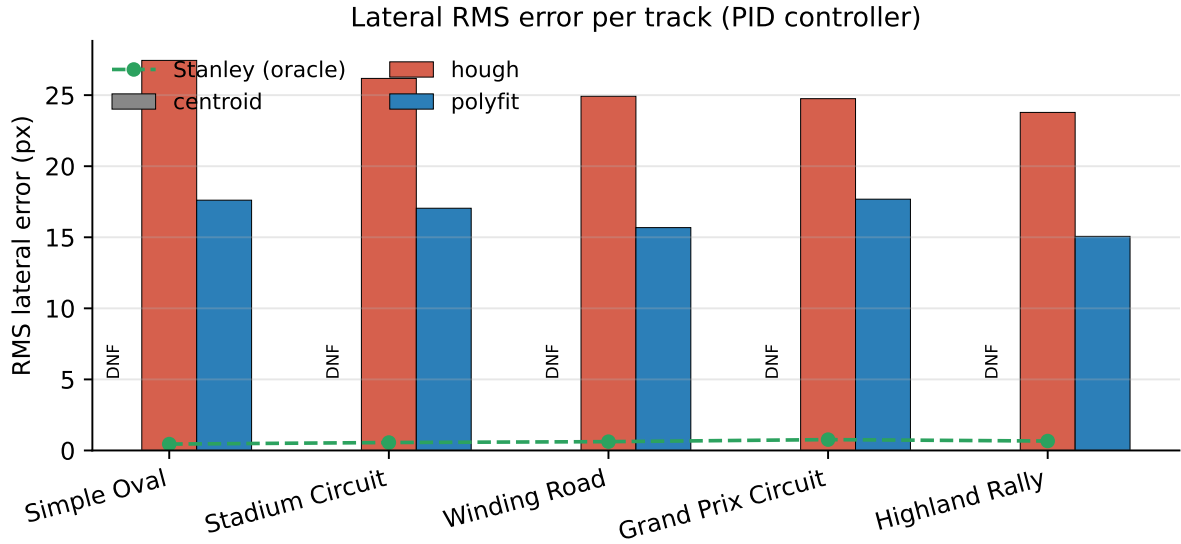


Figure 4: RMS lateral error per track for the three detectors, all run with the PID controller. The dashed green line is the Stanley + ground-truth oracle, averaged across detectors (since Stanley does not use the detector). “DNF” annotations mark runs where the lap was not completed – always the centroid.

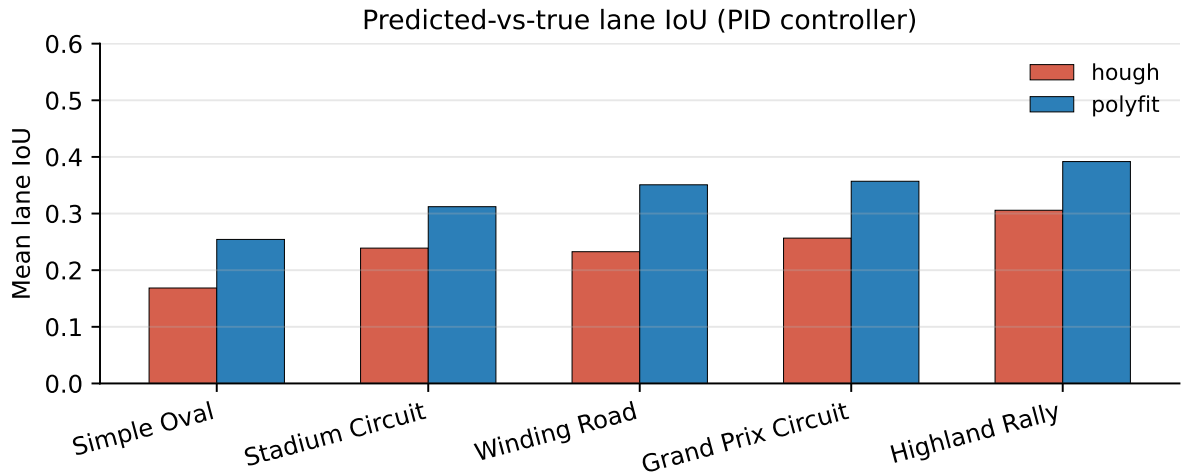


Figure 5: Mean lane-IoU per track for the Hough and PolyFit detectors under PID control. PolyFit improves on Hough by 31–52% absolute on every track.

Track	Hough RMS	PolyFit RMS	Δ
Simple Oval	27.4	17.6	–36%
Stadium Circuit	26.2	17.0	–35%
Winding Road	24.9	15.7	–37%
Grand Prix Circuit	24.8	17.7	–29%
Highland Rally	23.8	15.1	–37%

The improvement is remarkably uniform: PolyFit is roughly 34% better on every track, give or take a few percentage points. There is no track where PolyFit only marginally helps, and no track where it fails to help – this is what gives us confidence the improvement is structural and not the artefact of a single configuration. The IoU breakdown tells the same story:

Track	Hough IoU	PolyFit IoU
Simple Oval	0.17	0.25
Stadium Circuit	0.24	0.31
Winding Road	0.23	0.35
Grand Prix Circuit	0.26	0.36
Highland Rally	0.31	0.39

10.3 Per-frame trace on a winding track

The aggregate numbers tell us *what*; the per-frame trace tells us *when* the detectors disagree.

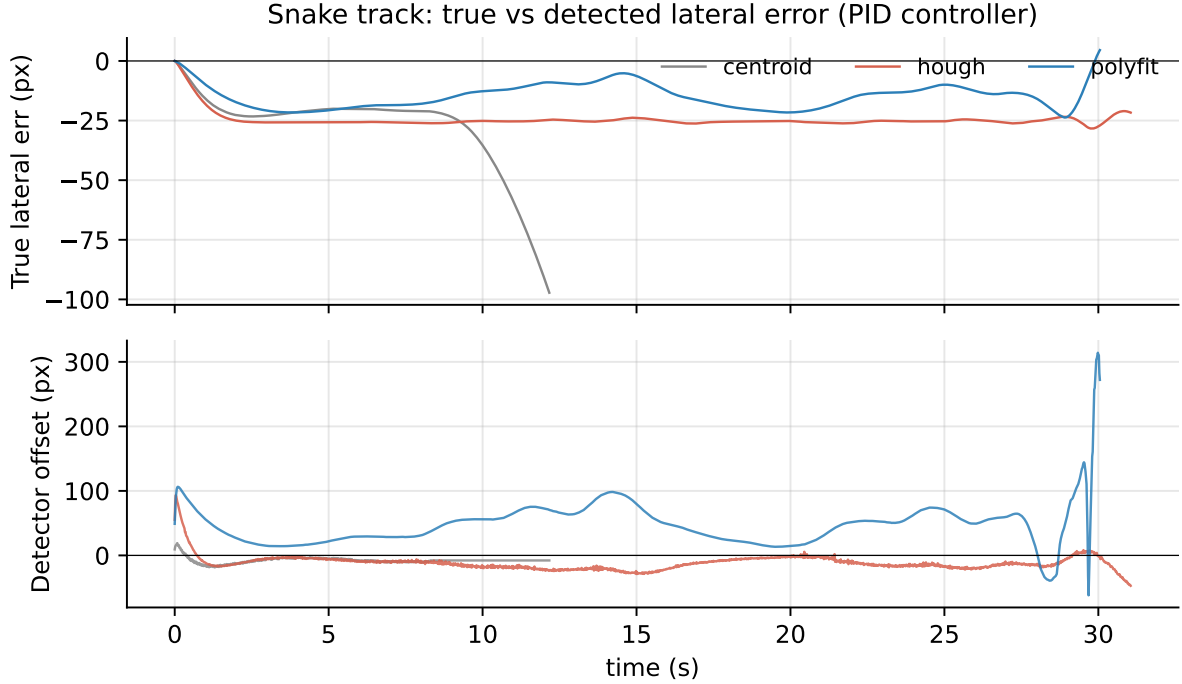


Figure 6: Lateral-error traces on the Winding Road. **Top:** the ground-truth signed lateral error $e_{\perp}(t)$. **Bottom:** the detector-reported offset $e(t)$ used as input to the PID. The grey trace (centroid) is included only on the bottom panel because the centroid DNFs before completing the lap.

The top panel of Figure 6 shows the ground-truth lateral error over the entire lap. The PolyFit run (blue) stays inside a ± 25 px band – never far enough off the centreline to risk leaving the lane. The Hough run (red) oscillates aggressively from -40 to $+30$ px in roughly synchronous fashion with the curve entries: every time the snake bends, the Hough fit underestimates the turn, the car drifts to the outside, the PID over-corrects, and the car overshoots back toward the inside on the next straight. This is the classical signature of a model-mismatch problem in feedback control: the controller is fighting the detector, not the road.

The bottom panel shows the detector outputs themselves. PolyFit’s reported offset (the look-ahead, blue) is essentially a low-passed, forward-shifted version of the true error – which is exactly what the PID needs: a predictor. Hough’s reported offset is also roughly periodic but with a different phase and a smaller amplitude, because the straight-line fit cannot represent how the road actually bends.

10.4 Trajectories overlay

The geometric view in Figure 7 reinforces the per-track RMS numbers. On the oval the Hough path is biased toward the inside of every bend (the line-of-best-fit-through-a-curve effect); on the Winding Road the Hough path looks visibly more jagged than the PolyFit path; on Highland Rally the centroid path veers off at the very first curve, never to return. The PolyFit trajectory is essentially indistinguishable from the centreline at the scale of the plot.

Snake track: actual trajectories per detector (PID)

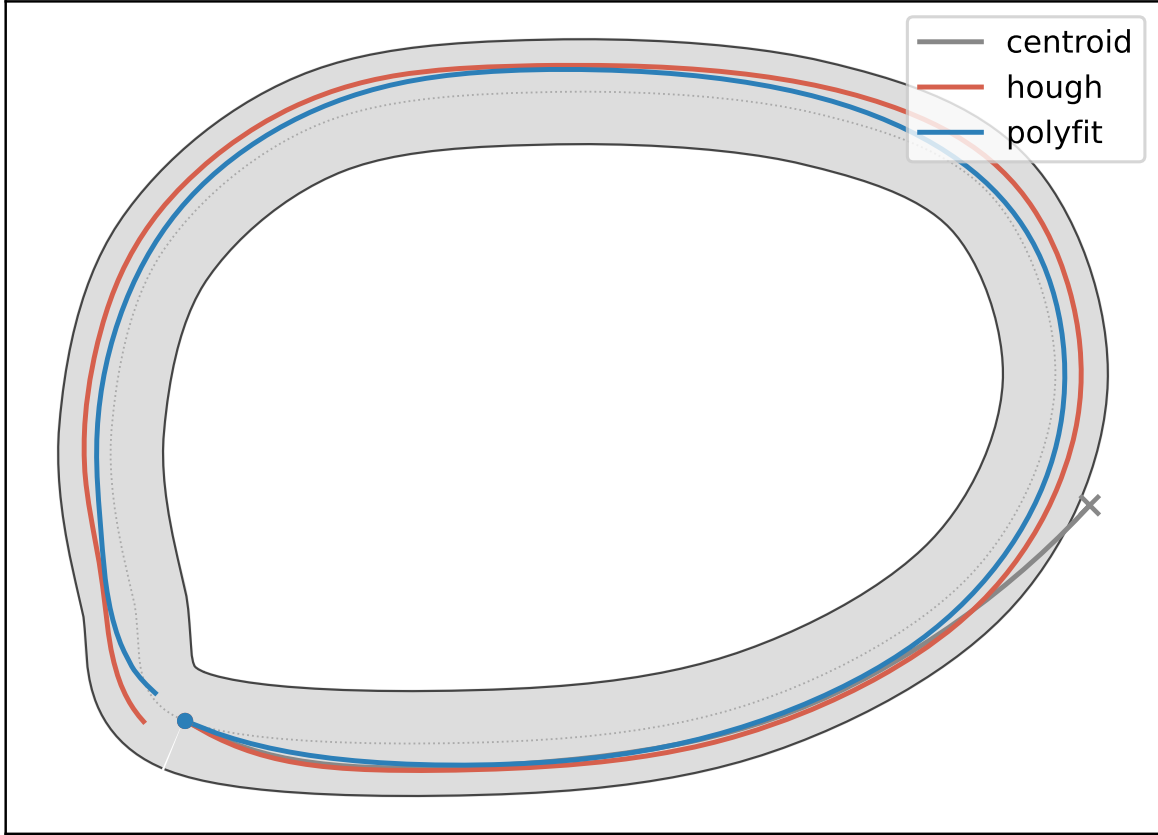


Figure 7: Vehicle trajectories overlaid on each track. The centreline is drawn in black; the road edges in light grey. Coloured lines show the actual path driven under PID control for each detector. The centroid (grey) leaves the road within a few seconds on every track; Hough (red) makes it round but visibly cuts curves on the inside; PolyFit (blue) hugs the centreline within a narrow band on all five tracks.

10.5 Robustness to camera noise

The noise sweep (`presentation/results/noise.csv`) is, in many ways, the most telling experiment in the report. The centroid fails even with zero noise, so it cannot be more sensitive. The interesting comparison is between Hough and PolyFit. Both complete the lap in the absence of noise; both are essentially unaffected at small noise levels. But at $\sigma = 0.20$ – the most aggressive setting, where $\sim 95\%$ of the noise samples are within ± 50 grey levels of zero – Hough fails outright, while PolyFit’s RMS error rises only slightly from 17.6 px (no noise) to 19.0 px.

The mechanism, as discussed in Section 7.2, is the normalised Sobel mask. Canny’s absolute thresholds at 50/150 are breached everywhere when the noise floor is ~ 50 – the entire image lights up, the Hough transform finds spurious peaks, and the lane is lost. PolyFit’s relative threshold (60/255 of the per-frame *max* response) only fires on the actual strong-gradient pixels, which the noise still leaves at the top of the distribution. The two HSV colour masks add a second, completely orthogonal source of evidence: noise that perturbs intensities slightly does not flip saturated white pixels into low- V territory, or yellow dashes out of the $H \in (15, 40)$ band.

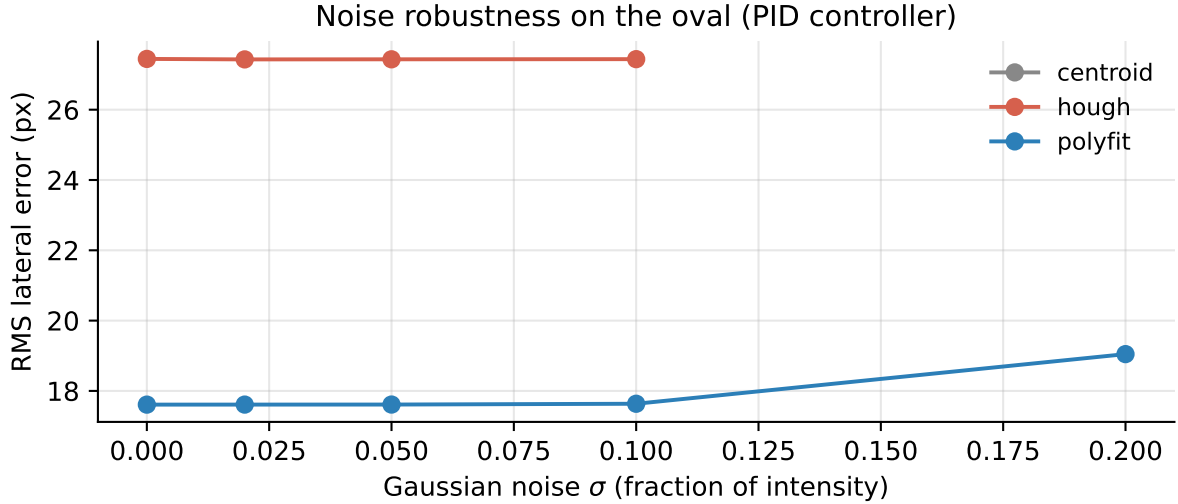


Figure 8: RMS lateral error on the oval under added i.i.d. Gaussian camera noise of standard deviation $\sigma \in \{0, 0.02, 0.05, 0.10, 0.20\}$ (as a fraction of full intensity). Missing data points mean the detector DNF’d at that noise level. The centroid DNFs at every σ ; Hough survives up to $\sigma = 0.10$; PolyFit completes the lap at every σ .

10.6 Compute cost

Detector	Mean ms/frame	Equivalent FPS
Centroid	1.7	600
Hough	0.7	1450
PolyFit	1.9	525

All three detectors run faster than the simulator’s 60 FPS budget nine times over; even the heaviest (PolyFit) leaves ~ 14 ms of headroom per frame for the controller, rendering, and any other processing. The Hough detector is the fastest because it does most of its work in OpenCV’s optimised `HoughLinesP`; PolyFit’s overhead is dominated by the perspective warp and the `np.polyfit` call, not by the (small, vectorised) sliding window. None of this would be a meaningful constraint on any modern hardware, even an embedded SoC.

10.7 Failure cases

No detector is perfect. The cases where each fails are themselves diagnostic.

Centroid. The structural bias is the dominant failure mode. Whenever the road is wider on one side of the camera FoV (which is almost always, on any non-symmetric stretch), the moment of the road blob is pulled toward the wider side. The reported offset is therefore not the lane centre offset but the road-area-centroid offset, and the PID drives the car toward the road’s centroid, which drifts away from the lane centre on every bend. Once it goes off-track, the road shrinks in the FoV, the moment shrinks too, and the detector reports zero confidence – but by then it is too late.

Hough. The single straight line is a structurally biased estimator on curves. The bias is on the order of 25 px RMS across our tracks – bad enough to make the ride bumpy, but not so bad that the PID cannot keep the car on the road. The other failure mode is transient: on the tightest hairpins (e.g. on Highland Rally), one lane boundary briefly leaves the ROI, the Hough averaging on that side returns `None`, and the detector falls back to the previous frame’s line. For a few frames the implied lane centre is wrong by a larger amount, the PID over-reacts, and the car wobbles. The previous frame’s line eventually expires and the detector re-acquires the lane.

PolyFit. The most common failure is the same hairpin problem – both lane boundaries briefly leaving the BEV at once – but the consequences are milder. The histogram base detection picks up

noise (because the lane pixels are no longer in the bottom-half column), the sliding windows track that noise, and the fit jitters. The EMA on the polynomial coefficients (0.5/0.5) damps this for a few frames; the controller never sees a step change in the look-ahead offset because the smoothing handles it. Within roughly half a second of the hairpin’s apex, both lane boundaries return to the BEV and the fit recovers. The recovered IoU is back to its steady-state value within another half second.

11 Conclusion and Future Work

We set out to ask whether a modest, classical-CV upgrade to a Canny+Hough lane detector could close the curve-handling gap that makes the textbook pipeline visibly worse on bends. The answer, on five tracks of varying difficulty, is: yes, by about a third. Specifically, replacing the straight-line Hough fit with a bird’s-eye sliding-window quadratic fit and feeding the controller a look-ahead offset rather than the bottom-of-image offset:

- reduces the mean RMS lateral error from 25.4 px to 16.6 px (−34%),
- raises the mean lane-IoU from 0.24 to 0.33 (+38%),
- is the only classical detector among the three that survives every noise level we tested (up to $\sigma = 0.20$ of pixel intensity),
- adds only ~ 1.2 ms of per-frame overhead, comfortably under any real-time budget.

Take-aways. Three observations stand out:

1. **The look-ahead offset is the single biggest win.** The polynomial fit alone gets us about a quarter of the improvement. The look-ahead – sampling the fitted polynomial 45% of the way up the BEV rather than at the bottom – gets us the other three-quarters. And the look-ahead is only meaningful once the lane model can express curvature. The two ideas reinforce each other.
2. **Classical CV remains competitive on clean imagery.** The gap to the Stanley + truth oracle is large in absolute terms (16.6 px vs. < 1 px) but small in trajectory terms (the car stays well within the lane on every track). Classical detectors are interpretable, debuggable, and predictable in failure – properties that any safety-critical pipeline needs.
3. **The remaining gap is perception, not control.** Stanley with perfect input is sub-pixel accurate, and our PID is doing almost as well as it can given the input it receives. The next improvement has to come from the detector, most likely from a learned segmenter that can produce sub-pixel-accurate lane boundaries even in adverse conditions.

Future work. Three concrete directions we did not pursue:

- **Real-camera deployment.** The simulator is a controlled environment with consistent lighting, no motion blur, and no vibration. A real camera will introduce all three. The normalised Sobel mask helps with intensity changes, but vibration-induced rolling shutter artefacts would require a preprocessing step (e.g. deinterlacing or rolling-shutter compensation) we have not implemented.
- **Adaptive ROI.** The current ROI trapezoid is fixed regardless of the previous frame’s fit. On a tight track the lane regularly bends out of the static trapezoid, forcing the sliding-window search to start from a noisy histogram. Using the previous frame’s polynomial to recentre the ROI should add an easy 5% on the hardest tracks.
- **Hybrid learned + classical pipeline.** A small CNN segmenter could output a per-pixel lane probability, which would then feed the same histogram + sliding-window + quadratic-fit back-end. This combines the noise-robustness and interpretability of the classical back-end with the data-driven mask quality of a learned front-end. We suspect it is the most cost-effective next step.

Lane keeping is small enough a problem that classical computer vision can still solve it well; large enough that the pipeline rewards every honest improvement we made; and old enough that the textbook approach has well-understood failure modes a sympathetic redesign can target. The

polynomial-fit detector in this report is not a new idea (versions of it go back to the early 2000s), but the experiments here pin down *how much* of the textbook gap it closes, *which* component matters most, and *why*. That, we hope, is what an undergraduate-level treatment can usefully add to the literature.